



PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG THÔNG TIN

Chương 2: Các khái niệm cơ bản về đối tượng và các biểu đồ UML

ThS. Nguyễn Văn Hữu Hoàng

I NỘI DUNG

Chương này tập trung trình bày một số chủ đề sau đây:

- 2.1. Một số khái niệm cơ bản của Hệ hướng đối tượng
- 2.2. Giới thiệu ngôn ngữ mô hình hóa thống nhất UML
- 2.3. Các biểu đồ trong UML

| 2.1 MỘT SỐ KHÁI NIỆM CƠ BẢN CỦA HỆ HƯỚNG ĐỐI TƯỢNG

❖ 2.1.1 Lớp và đối tượng?

- Lớp (**class**) là một thuật ngữ để xác định một tập hợp các thể hiện hay các đối tượng có các đặc trưng chung nào đó. Lớp đóng gói các đặc điểm chung của một nhóm các đối tượng.
- Đối tượng (**object**) là một thể hiện của một lớp.
- Đối tượng có thể là một sự vật, một thực thể, một danh từ hoặc bất cứ cái gì mà bạn có thể nhặt lên hoặc ném đi, hay những gì mà bạn có thể tưởng tượng ra với một số đặc tính nào đó của nó.
- Mỗi đối tượng có các thuộc tính (**attribute**) mô tả thông tin về đối tượng đó.
- Trạng thái (**state**) của một đối tượng được xác định bởi bộ giá trị của những thuộc tính và quan hệ với những đối tượng khác tại một thời điểm cụ thể.
- Hành vi (**behavior**) hay hành động (**action**), thao tác (**operation**), phương thức (**method**) nhằm đặc tả những gì đối tượng này có thể thực hiện được.

| Biểu diễn đối tượng và lớp

- Lớp và đối tượng sẽ được biểu diễn theo ngôn ngữ mô hình hóa thống nhất UML.
- Quy ước:
 - Tên lớp: Danh từ bắt đầu bằng chữ hoa.
 - Thuộc tính, phương thức: Bắt đầu bằng chữ thường.
- Ví dụ:

Sach
ten
tacgia
nhaXuatban
namXuatban
xoaSach ()
themSach ()
capnhatThongtinSach ()

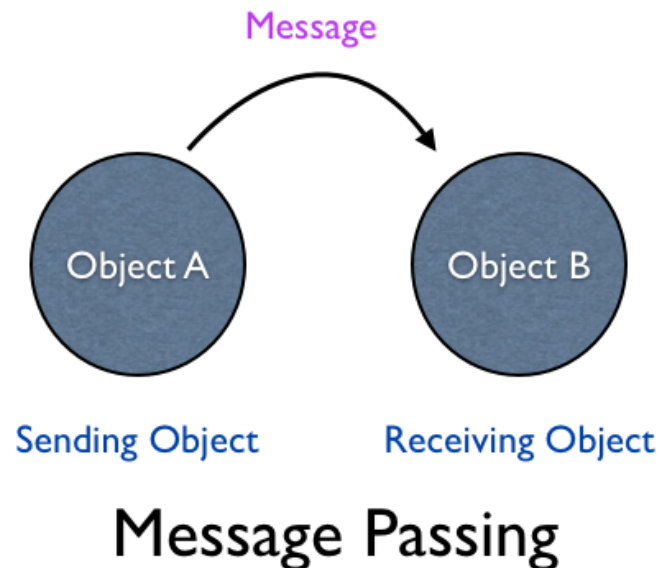
Biểu diễn lớp

<u>Sach1: Sach</u>
ten = PhantichvaThietkeHuongdoituong
tacgia = DangVanDuc
nhaXuatban = Giaoduc
namXuatban = 2002
xoaSach ()
themSach ()
capnhatThongtinSach ()

Biểu diễn đối tượng

| 2.1.2 Phương thức và thông điệp

- Phương thức (**method**) cài đặt hành vi (hay còn gọi hành động, thao tác) của đối tượng. Phương thức chính là hành động mà một đối tượng có thể thực hiện.
- Thông điệp (**message**) là thông tin được gửi cho đối tượng để kích hoạt các phương thức. Một thông điệp chính là lời gọi hàm hay thủ tục truyền thống nhưng lại từ một đối tượng này đến đối tượng khác.

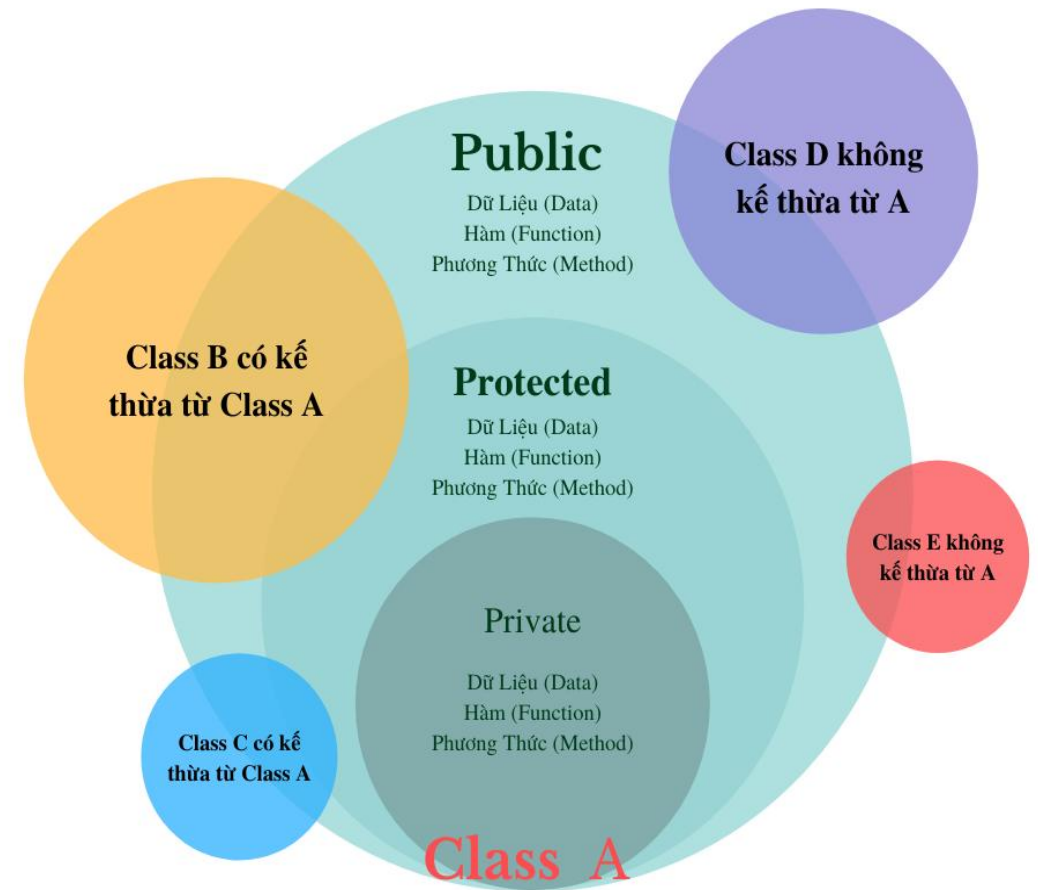


| 2.1.3 Đặc tính của lập trình hướng đối tượng (OOP)

- Tính đóng gói (**Encapsulation**)
- Tính kế thừa (**Inheritance**)
- Tính đa hình (**Polymorphism**)
- Tính trừu tượng (**Abstraction**)

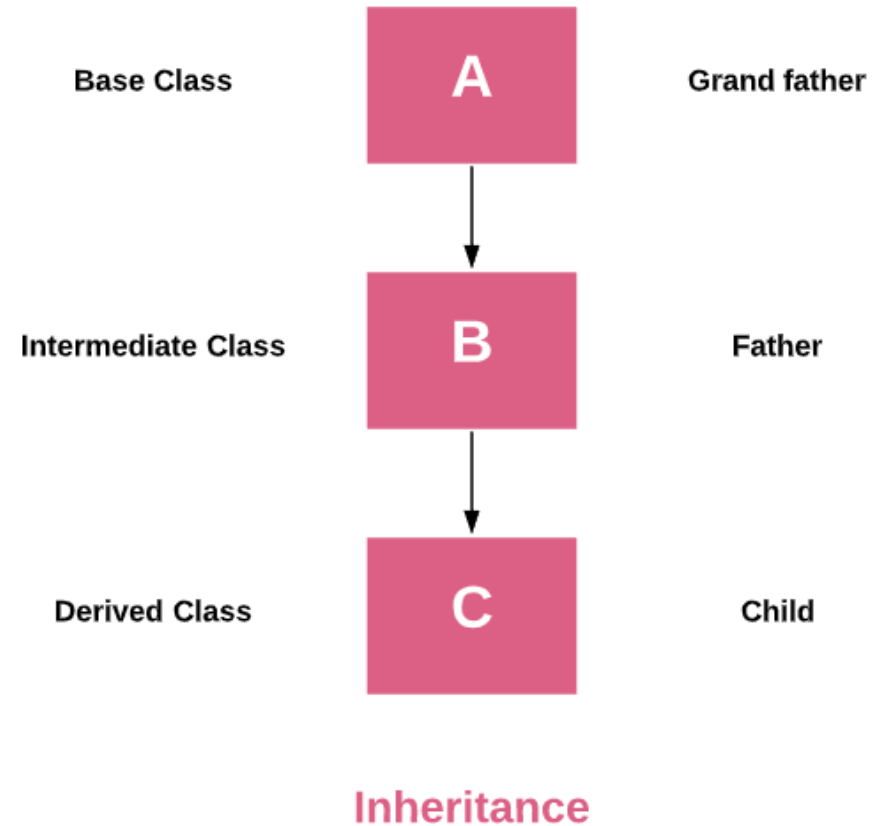
| Tính đóng gói (Encapsulation)

- Tính đóng gói được hiểu đơn giản các dữ liệu (data) và các hàm/phương thức (functions/methods) có liên quan đến nhau được “đóng gói” thành các lớp (class) để tiện cho việc quản lý và sử dụng.



| Tính kế thừa (Inheritance)

- Tính kế thừa trong lập trình hướng đối tượng thể hiện mối quan hệ **cha-con** giữa các lớp (classes) với nhau và nhờ vào mối quan hệ này cho phép lớp con thừa hưởng những đặc tính của lớp cha.



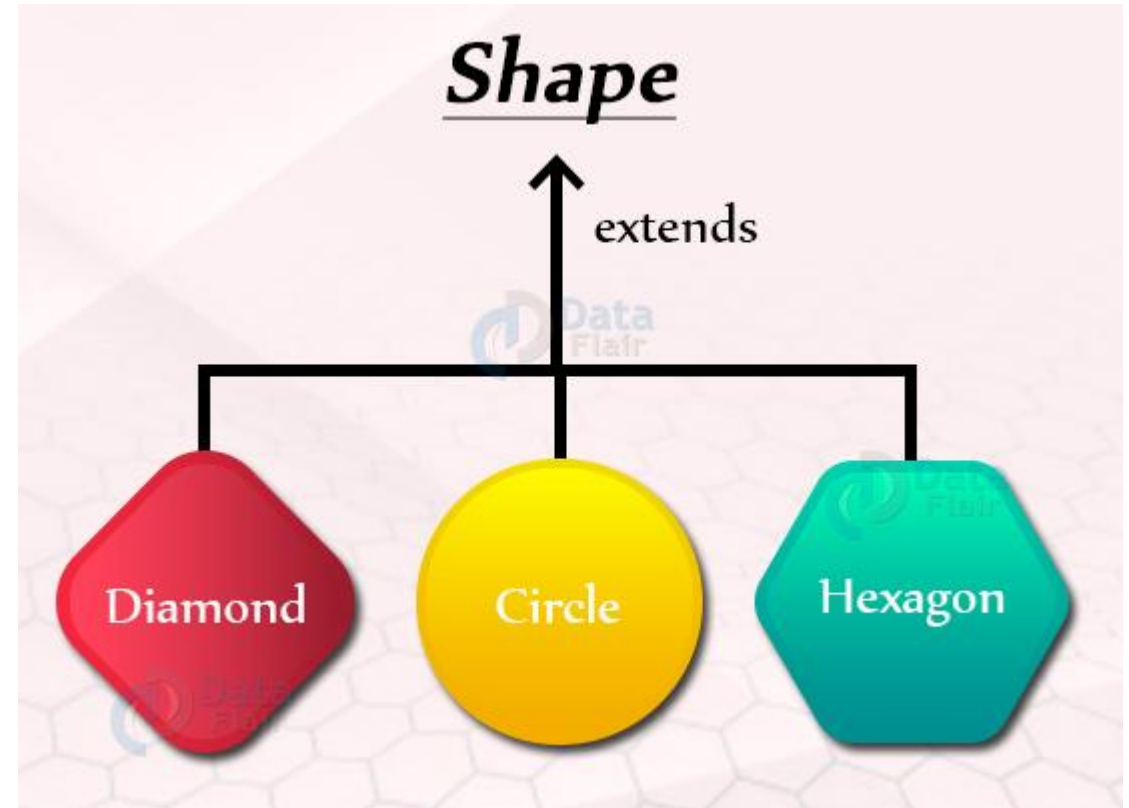
| Tính đa hình (Polymorphism)

- **Tính đa hình** (Polymorphism) là một đối tượng có thể thực hiện một tác vụ theo nhiều cách khác nhau.
- Nó cho phép thực hiện cùng một hành động trên các đối tượng khác nhau mà không cần quan tâm đến loại đối tượng cụ thể.
- Giúp tái sử dụng mã nguồn, linh hoạt và phân cấp, dễ bảo trì.



| Tính trừu tượng (Abstraction)

- **Tính trừu tượng** là khái niệm chỉ “hiển thị” các thuộc tính cần thiết và “ẩn” thông tin không cần thiết.
- Mục đích chính của trừu tượng là che giấu những chi tiết không cần thiết khỏi người dùng, giảm sự phức tạp của bài toán.
- Che dấu thông tin vì vậy đảm bảo được sự độc lập của từng phần của bài toán, chia bài toán thành từng phần trừu tượng khác nhau và giải quyết bài toán trên từng mức trừu tượng đó.

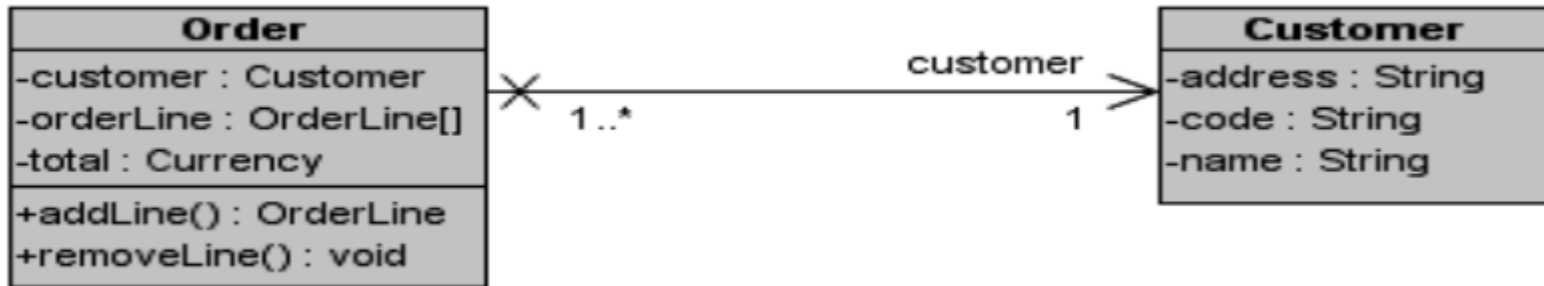


| 2.1.4 Quan hệ giữa các lớp

- Không có một đối tượng nào có thể tồn tại và hoạt động riêng lẻ. Tất cả các hành động của đối tượng đều liên quan tới các đối tượng khác một cách trực tiếp hoặc gián tiếp, mạnh hoặc yếu.
- Trong mô hình hóa đối tượng với UML, quan hệ giữa các đối tượng được thể hiện theo ba cách chính sau đây:
 - Liên kết.
 - Kết tập.
 - Hợp thành.

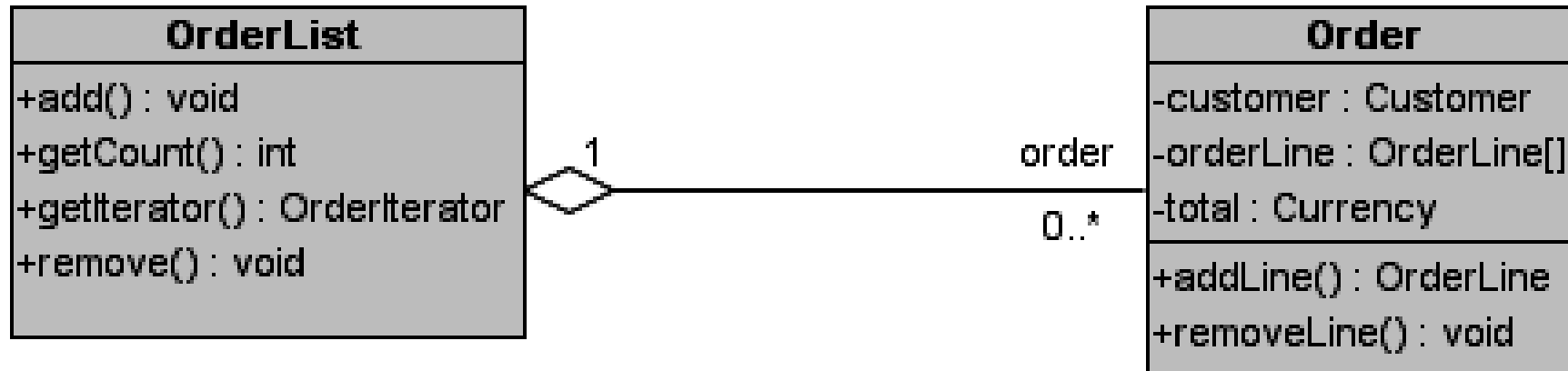
| Liên kết (association)

- Là một dạng quan hệ giữa hai đối tượng hay lớp đối tượng có liên quan và phụ thuộc vào nhau. Có hai dạng liên kết một chiều và hai chiều tùy theo đối tượng phụ thuộc nhau một chiều hay cả hai chiều.
- Ví dụ *Order* liên kết 1 chiều đến *Customer* để lấy thông tin Khách hàng.



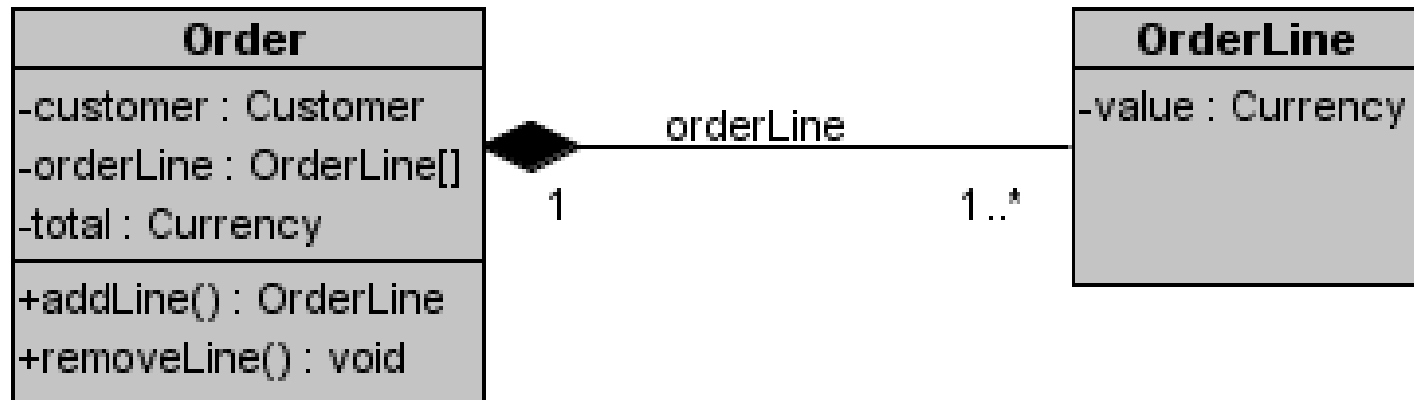
| Kết tập (aggregation)

- Kết tập nghĩa là một đối tượng là một bộ phận của đối tượng kia, các đối tượng kết hợp với nhau để tạo thành một đối tượng lớn hơn.
- Kết tập thường có dạng phân cấp **bộ phận-toàn thể** nghĩa là ám chỉ sự phụ thuộc giữa bộ phận và toàn thể
- Ví dụ: Quan hệ giữa lớp *OrderList* và lớp *Order* là thuộc kiểu kết hợp, một danh sách *OrderList* bao gồm nhiều *Order* nhưng các *Order* có thể tồn tại độc lập mà không cần phải là một bộ phận của danh sách *OrderList* cụ thể.



I Hợp thành (Composition)

- Hợp thành là dạng mạnh hơn của kết tập trong đó bộ phận phụ thuộc vào toàn thể một cách duy nhất và đối tượng toàn thể có trách nhiệm tạo lập và hủy bỏ đối tượng bộ phận. Như vậy, khi đối tượng toàn thể bị hủy thì đối tượng bộ phận cũng hủy theo.
- Ví dụ: Quan hệ giữa *Order* và *OrderLine* là thuộc kiểu hợp thành. Các mặt hàng mà khách hàng đặt *OrderLine* trong một đơn hàng *Order* đều thuộc về *Order* và không có ý nghĩa bên ngoài *Order* đó. *Order* có trách nhiệm hoàn toàn trong việc tạo, quản lý và xóa bất kỳ *OrderLine* nào trong *Order* đó.



I MỤC ĐÍCH CỦA PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

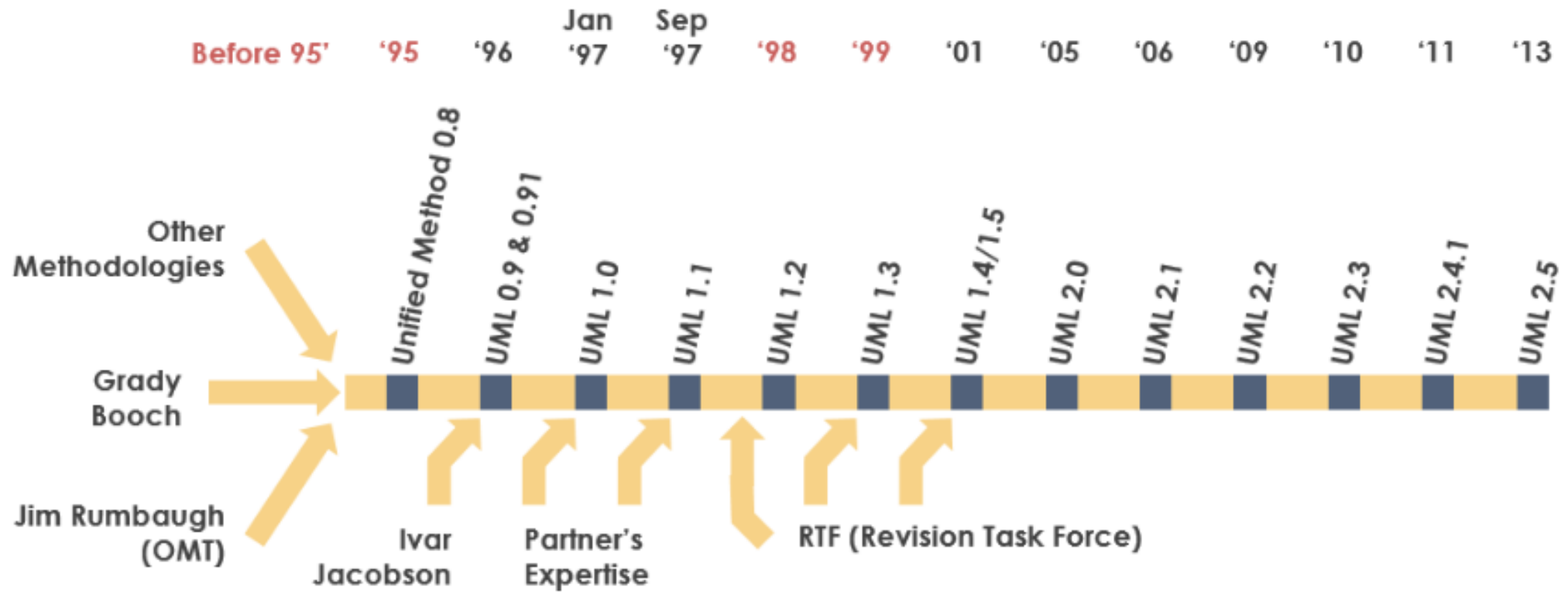
- Chuyển các yêu cầu của bài toán thành một **bản thiết kế** của hệ thống sẽ được xây dựng.
- Tập trung vào quá trình phân tích các **YÊU CẦU** của hệ thống và thiết kế các **MÔ HÌNH** cho hệ thống đó trước giai đoạn lập trình.
- Nhằm đảm bảo mục đích và yêu cầu của hệ thống được ghi lại một cách hợp lý trước khi hệ thống được xây dựng.
- Cung cấp cho người dùng, khách hàng, kỹ sư phân tích, thiết kế nhiều cái nhìn khác nhau về cùng một hệ thống.

I 2.2 GIỚI THIỆU NGÔN NGỮ MÔ HÌNH HÓA UML

2.2.1 Giới thiệu về UML

- Các ngôn ngữ lập trình hướng đối tượng ra đời khá sớm, ví dụ như Simula-67 (năm 1967), Smalltalk (đầu những năm 1980), C++, CLOS (giữa những năm 1980)... Tuy nhiên, mãi cho đến năm 1995, các nhóm nghiên cứu mới tập trung xem xét những phương pháp luận và ngôn ngữ mô hình với ký hiệu khác nhau, như *Booch* của *Grady Booch*, *OMT* của *James Rumbaugh*, *OOSE* của *Ivar Jacobson*, hay *OOA & OOD* của *Coad* và *Yordon*.
- Vấn đề làm thế nào xây dựng được một ngôn ngữ mô hình hóa thống nhất như một chuẩn cho những người phát triển phần mềm hướng đối tượng đã trở thành nhu cầu cấp thiết?
- James Rumbaugh, Grady Booch và Ivar Jacobson đã cùng nhau xây dựng một *Ngôn ngữ Mô hình hoá thống nhất UML (Unified Modeling Language)* đầu tiên được đưa ra năm 1997 và sau đó được chuẩn hoá để trở thành phiên bản 1.0.
- Phiên bản 2.0 với nhiều công cụ hỗ trợ đã được sử dụng rộng rãi trong công nghiệp phần mềm.
- UML đã không ngừng phát triển cho đến nay với phiên bản 2.5 (chi tiết tham khảo <http://www.omg.org/spec/UML/>)

| LỊCH SỬ PHÁT TRIỂN CỦA UML



| 2.2.2 Một số đặc điểm của ngôn ngữ mô hình hoá UML

- **UML** là ngôn ngữ mô hình hoá được xây dựng để đặc tả, phát triển và viết tài liệu cho các hệ phần mềm hướng đối tượng. Nó bao gồm một tập các khái niệm, các ký hiệu, các biểu đồ và hướng dẫn sử dụng.
- Mục đích của ngôn ngữ UML là:
 - Mô hình hoá các hệ thống bằng cách sử dụng các khái niệm hướng đối tượng.
 - Thiết lập sự liên hệ từ nhận thức của con người đến các sự kiện cần mô hình hoá.
 - Giải quyết vấn đề về mức độ thừa kế trong các hệ thống phức tạp với nhiều ràng buộc khác nhau.
 - Tạo một ngôn ngữ mô hình hoá có thể sử dụng được bởi người và máy.
- UML hỗ trợ phân rã hệ hướng đối tượng dựa trên cấu trúc tĩnh và hành vi động của hệ thống.
 - Các cấu trúc tĩnh (**static structure**) xác định các kiểu đối tượng quan trọng của hệ thống và mối quan hệ giữa các đối tượng đó nhằm đến cài đặt sau này.
 - Các hành vi động (**dynamic behavior**) xác định các hành động của các đối tượng theo thời gian và tương tác giữa các đối tượng.

I Những đặc điểm mô tả khái quát về UML

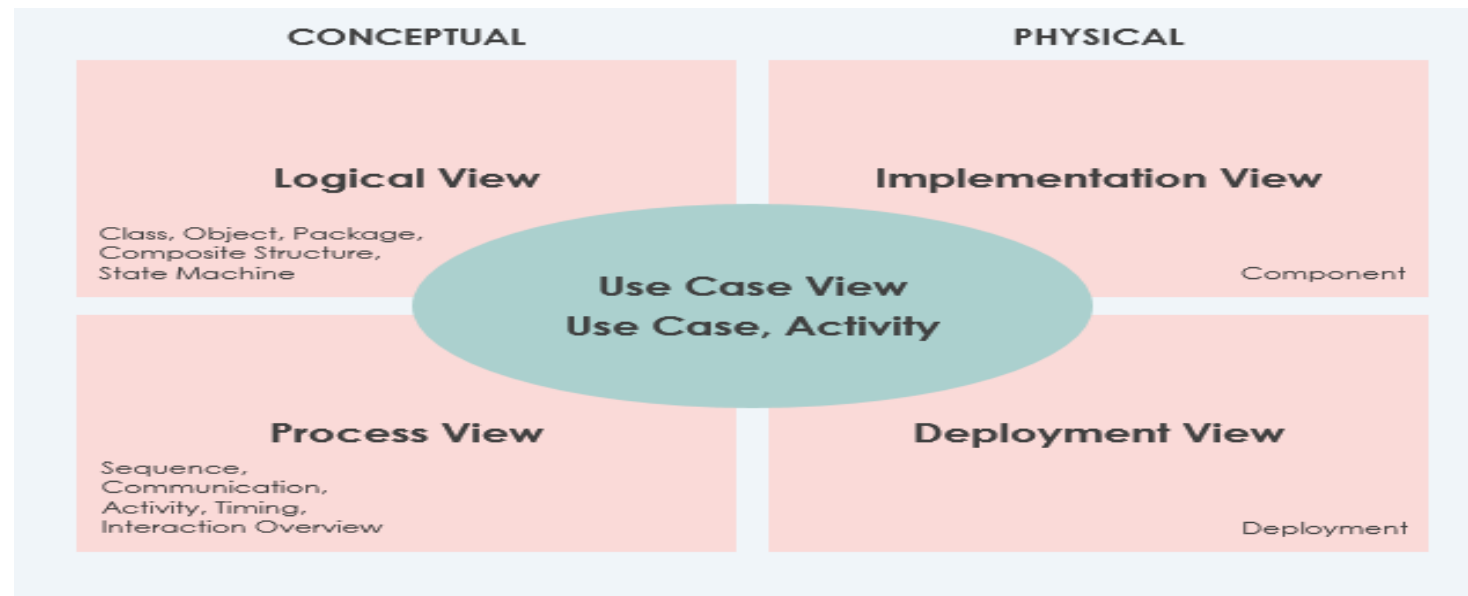
- **Khung nhìn:** là một trừu tượng hóa để thể hiện những khía cạnh khác nhau của hệ thống. Mỗi khung nhìn gồm một số biểu đồ và các khung nhìn khác nhau sẽ cung cấp một bức tranh đầy đủ về hệ thống.
- **Biểu đồ:** các biểu đồ bao gồm các phần tử là đồ thị mô tả các nội dung theo khung nhìn. UML 2.4 có khoảng 14 kiểu biểu đồ khác nhau và tùy theo hệ thống khi kết hợp một số biểu đồ này sẽ cho ta toàn bộ khung nhìn hệ thống.
- **Các phần tử mô hình:** các khái niệm dùng trong biểu đồ là những phần tử mô hình nhằm biểu diễn các khái niệm hướng đối tượng quen thuộc như lớp, đối tượng, thông điệp và các quan hệ như tổng quát hóa, liên kết, phụ thuộc, kết hợp, hợp thành.
- **Cơ chế tổng quát:** các cơ chế tổng quát cung cấp cách mở rộng UML cho các miền đặc biệt và cách thêm chú thích, thông tin, ngữ nghĩa cho phần tử mô hình.
- **Kiến trúc hướng mô hình:** Model-Driven Architecture cung cấp các hướng dẫn để cấu trúc các đặc tả lại thành các mô hình, tách biệt giữa thiết kế và kiến trúc. Thiết kế tập trung vào các yêu cầu chức năng (use case) còn các kiến trúc thì tập trung vào thượng tầng kiến trúc để đáp ứng các yêu cầu phi chức năng.

| Các khung nhìn trong UML

- **Khung nhìn ca sử dụng:** Khung nhìn này thể hiện chức năng của hệ thống đối với tác nhân bên ngoài. Tác nhân có thể là con người hay hệ thống khác. Khung nhìn ca sử dụng được sử dụng bởi *khách hàng, nhân viên thiết kế, nhân viên phát triển và kiểm thử*. Nó được mô tả bởi biểu đồ ca sử dụng và đôi khi cùng với biểu đồ hoạt động. Mỗi một ca sử dụng là một mô tả tổng quát chức năng yêu cầu. Khung nhìn ca sử dụng là trung tâm và định hướng các khung nhìn khác.
- **Khung nhìn logic:** Khung nhìn này thể hiện cách thiết kế chức năng bên trong hệ thống theo cấu trúc và hành vi. Khung nhìn này chủ yếu dành cho *người thiết kế và phát triển phần mềm*. Nó mô tả cấu trúc tĩnh (lớp, đối tượng và quan hệ) và mô hình động qua tương tác bằng phương thức giữa các đối tượng. Cấu trúc tĩnh được mô tả bởi các biểu đồ lớp và đối tượng; mô hình động được mô tả bởi máy trạng thái, biểu đồ tương tác và biểu đồ hoạt động.
- **Khung nhìn cài đặt:** Khung nhìn này thể hiện cách tổ chức mã nguồn và mã thực thi. Nó mô tả những môđun chính và phụ thuộc của chúng. Nó chủ yếu dành cho *người phát triển* và bao gồm các sản phẩm phần mềm như các môđun mã nguồn với cấu trúc và phụ thuộc hay thông tin quản lý và báo cáo tiến độ...

| Các khung nhìn trong UML (tt)

- **Khung nhìn tiến trình:** Khung nhìn này chỉ ra các phần tử chính như thông lượng, khả năng mở rộng...trong hệ thống liên quan đến hiệu năng của tiến trình. Nó liên quan đến tính phi chức năng của hệ thống. Khung nhìn này thể hiện bởi các biểu đồ động (máy trạng thái, các biểu đồ hoạt động và tương tác), biểu đồ cài đặt (các biểu đồ tương tác và triển khai) và biểu đồ thời điểm.
- **Khung nhìn triển khai:** Khung nhìn này thể hiện việc triển khai hệ thống trên các máy tính và các thiết bị. Nó được sử dụng bởi các *nhân viên phát triển, tích hợp và kiểm thử*. Khung nhìn này được thể hiện bởi biểu đồ triển khai.



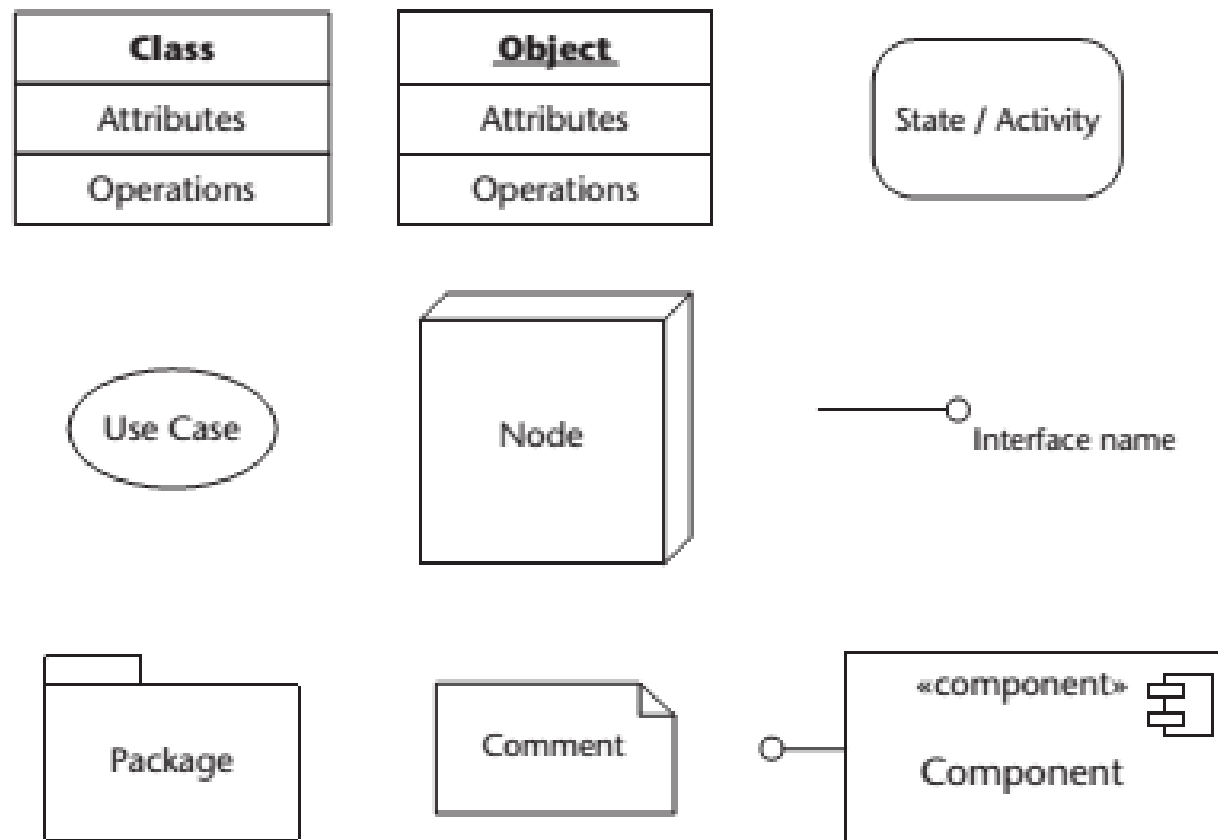
| 2.2.3 Các khái niệm cơ bản trong UML

- **Mô hình (model)** là một biểu diễn của sự vật, đối tượng hay một tập các sự vật trong một lĩnh vực ứng dụng theo một quan điểm nào đó.
- Mục đích của mô hình:
 - Mô tả các đặc điểm hay khía cạnh quan trọng của sự vật cần quan tâm và biểu diễn theo một tập ký hiệu hoặc quy tắc nào đó. Do đó, cùng một đối tượng nhưng trong hai hệ thống khác nhau có thể có các đặc điểm cần quan tâm khác nhau.
 - Thể hiện các mô tả bởi những biểu thức toán học hoặc bởi các biểu đồ dựa trên một tập ký hiệu và quy tắc xác định. Trong hệ phần mềm hướng đối tượng, các biểu đồ UML được sử dụng rộng rãi để mô hình hệ thống.

| Các phần tử mô hình

Ký hiệu:

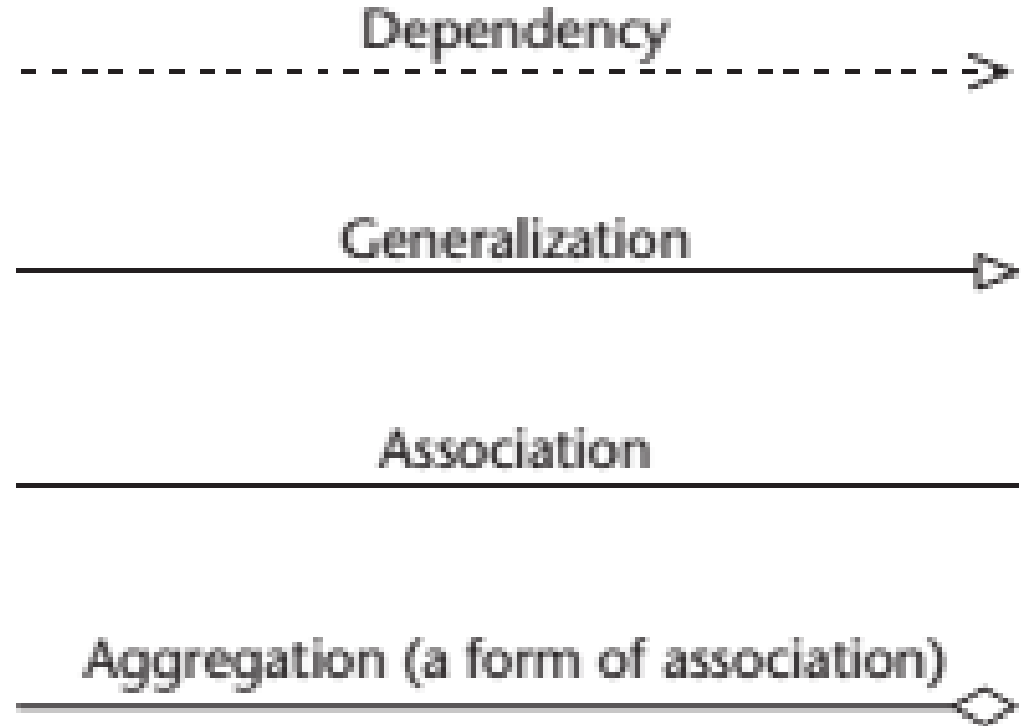
- Một số ký hiệu để mô hình hóa hướng đối tượng thường gặp trong UML như **lớp** (class) và **đối tượng** (object) được mô tả bởi một hộp với 3 gian là tên lớp, **thuộc tính** (attributes), **thao tác** (operations) trong đó thể hiện đối tượng có gạch chân.
- **Ca sử dụng** (use case) thể hiện bởi hình quả trám gắn với tên ca sử dụng.
- **Trạng thái** (state) hay **hoạt động** (activity) biểu diễn bởi hình chữ nhật góc tròn.



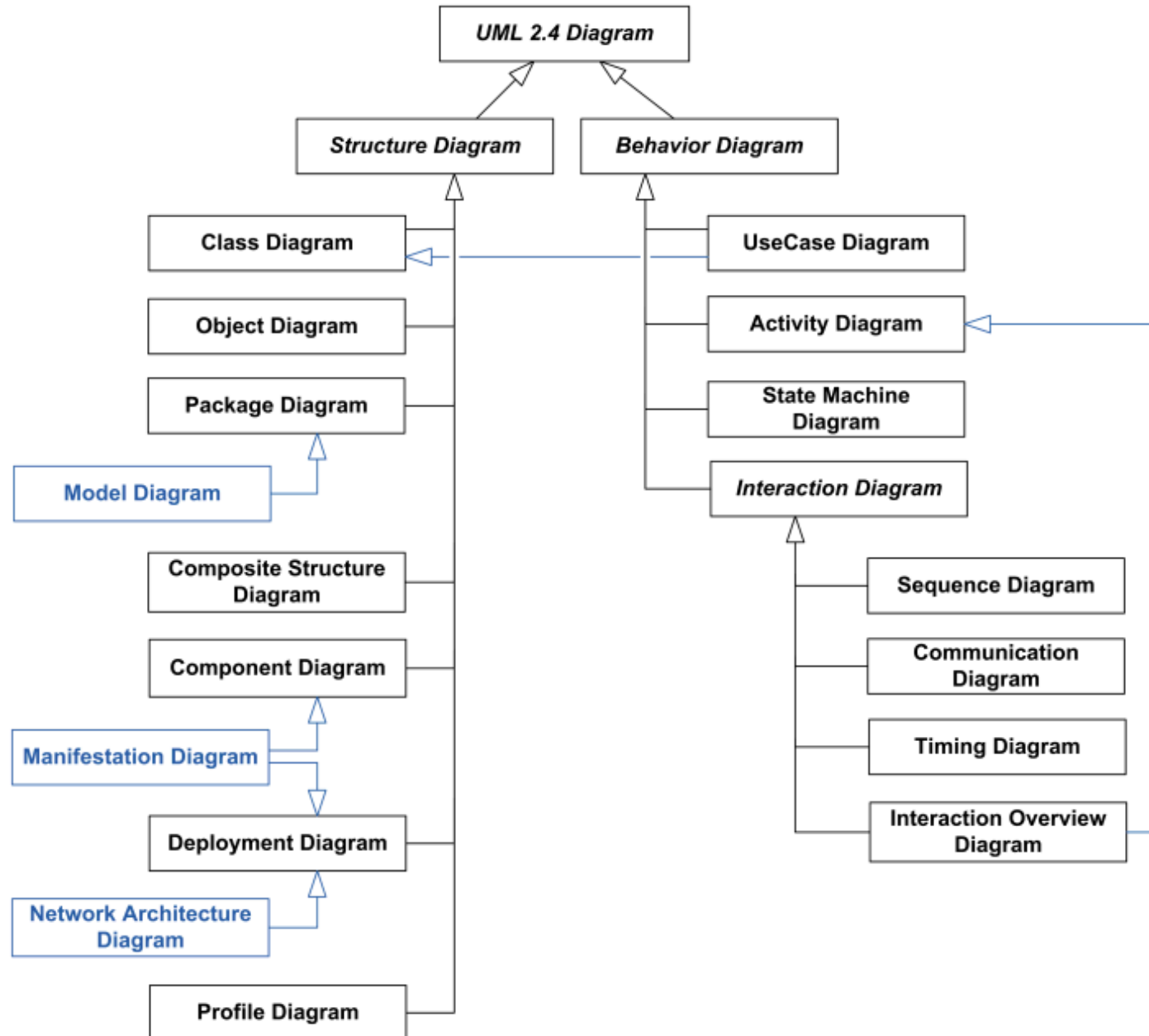
| Các phần tử mô hình

Quan hệ:

- Các quan hệ này có thể xuất hiện trong bất cứ mô hình nào của UML dưới các dạng khác nhau như quan hệ giữa các ca sử dụng, quan hệ trong biểu đồ lớp...
- Một số các quan hệ: phụ thuộc (dependency), tổng quát hóa (generalization), liên kết (association) và kết hợp (aggregation).



| 2.3 CÁC BIỂU ĐỒ TRONG UML



| Biểu đồ cấu trúc

- **Biểu đồ cấu trúc** (Structure Diagram): Nhóm các biểu đồ này biểu diễn các cấu trúc tĩnh của hệ phần mềm cần được mô hình hoá. Các biểu đồ trong mô hình tĩnh tập trung biểu diễn khía cạnh tĩnh liên quan đến cấu trúc cơ bản cũng như các phần tử chính của hệ thống, gồm:
 - Biểu đồ lớp (class diagram)
 - Biểu đồ đối tượng (object diagram)
 - Biểu đồ thành phần (component diagram)
 - Biểu đồ gói (package diagram)
 - Biểu đồ triển khai (deployment diagram)
 - Biểu đồ cấu trúc phức hợp (composite structure diagram)
 - Biểu đồ gói mở rộng (profile package)

| Biểu đồ hành vi

- **Biểu đồ hành vi** (Behavior Diagram): Nhóm biểu đồ này nhằm thể hiện các hoạt động và hành vi của hệ thống, cũng như tương tác giữa các phần tử bên trong và bên ngoài hệ thống, gồm:
 - Biểu đồ ca sử dụng (use case diagram)
 - Biểu đồ hoạt động (activity diagram)
 - Biểu đồ tuần tự (sequence diagram)
 - Biểu đồ giao tiếp (communication diagram)
 - Biểu đồ trạng thái (state machine diagram)
 - Biểu đồ tương tác tổng quát (interaction overview diagram)
 - Biểu đồ thời khắc (timing diagram)

| 2.3.1 Biểu đồ ca sử dụng

❖ Ý nghĩa:

- Biểu đồ ca sử dụng (use case diagram) biểu diễn các chức năng của hệ thống và thể hiện sự tương tác giữa các **tác nhân** và hệ thống cần xây dựng. Nó bao gồm một tập các **tác nhân** (actor), các **ca sử dụng** (use case) và các mối quan hệ giữa các ca sử dụng. Tác nhân có thể là con người, một hệ thống khác cung cấp thông tin hay tác động tới hệ thống.

❖ Cách xây dựng:

- Biểu đồ ca sử dụng có thể được phân rã theo nhiều mức khác nhau. Từ tập yêu cầu xác định được, biểu đồ ca sử dụng sẽ chỉ ra hệ thống cần thực hiện điều gì để thoả mãn các yêu cầu của người dùng hệ thống đó. Đi kèm với biểu đồ ca sử dụng là các **kịch bản** (scenario) nhằm mô tả chi tiết quá trình thực hiện ca sử dụng đó.
- Mô hình chức năng với biểu đồ ca sử dụng được xây dựng qua quá trình thảo luận giữa bên phát triển và khách hàng/người sử dụng. Biểu đồ này được xem là một thỏa thuận về đặc tả yêu cầu giữa các bên liên quan.

I Tập ký hiệu UML cho biểu đồ ca sử dụng

- **Hệ thống**: hệ thống biểu diễn ranh giới giữa bên trong và bên ngoài của một chủ thể trong phần mềm chúng ta xây dựng.
- **Tác nhân** (actor): là người dùng của hệ thống, một tác nhân có thể là một người dùng thực hoặc các hệ thống máy tính khác có vai trò nào đó trong hoạt động của hệ thống.
- **Các ca sử dụng**: đây là thành phần cơ bản của biểu đồ ca sử dụng. Các ca sử dụng được biểu diễn bởi các hình elip thể hiện một chức năng xác định của hệ thống.

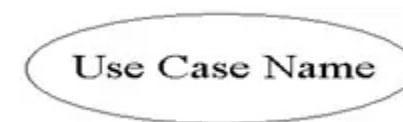
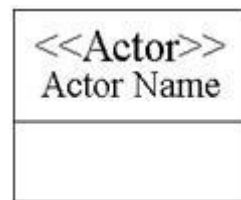


Biên của hệ thống



Ký hiệu của Tác nhân

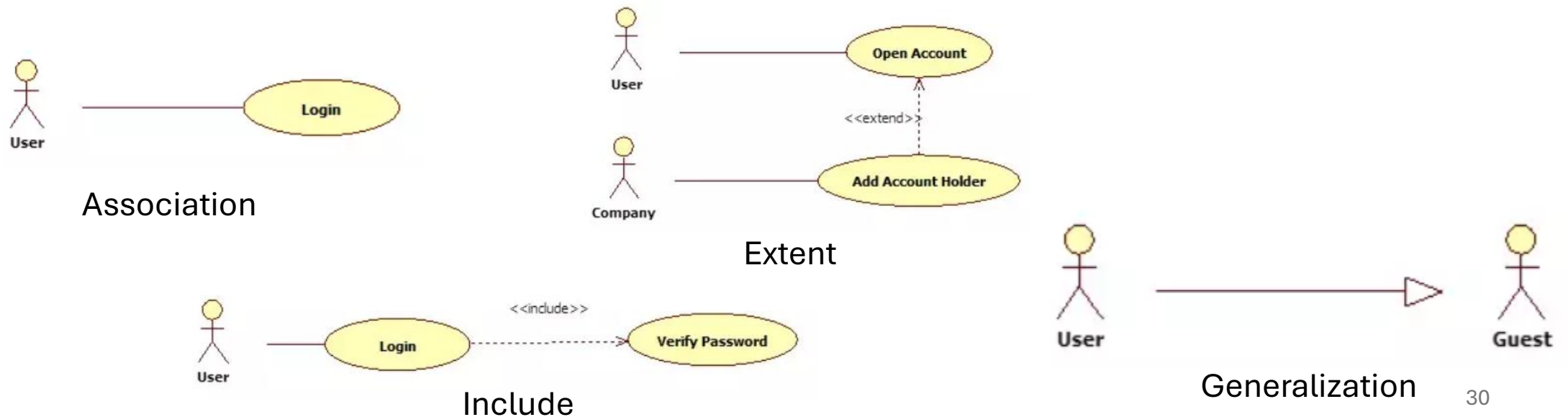
Hoặc



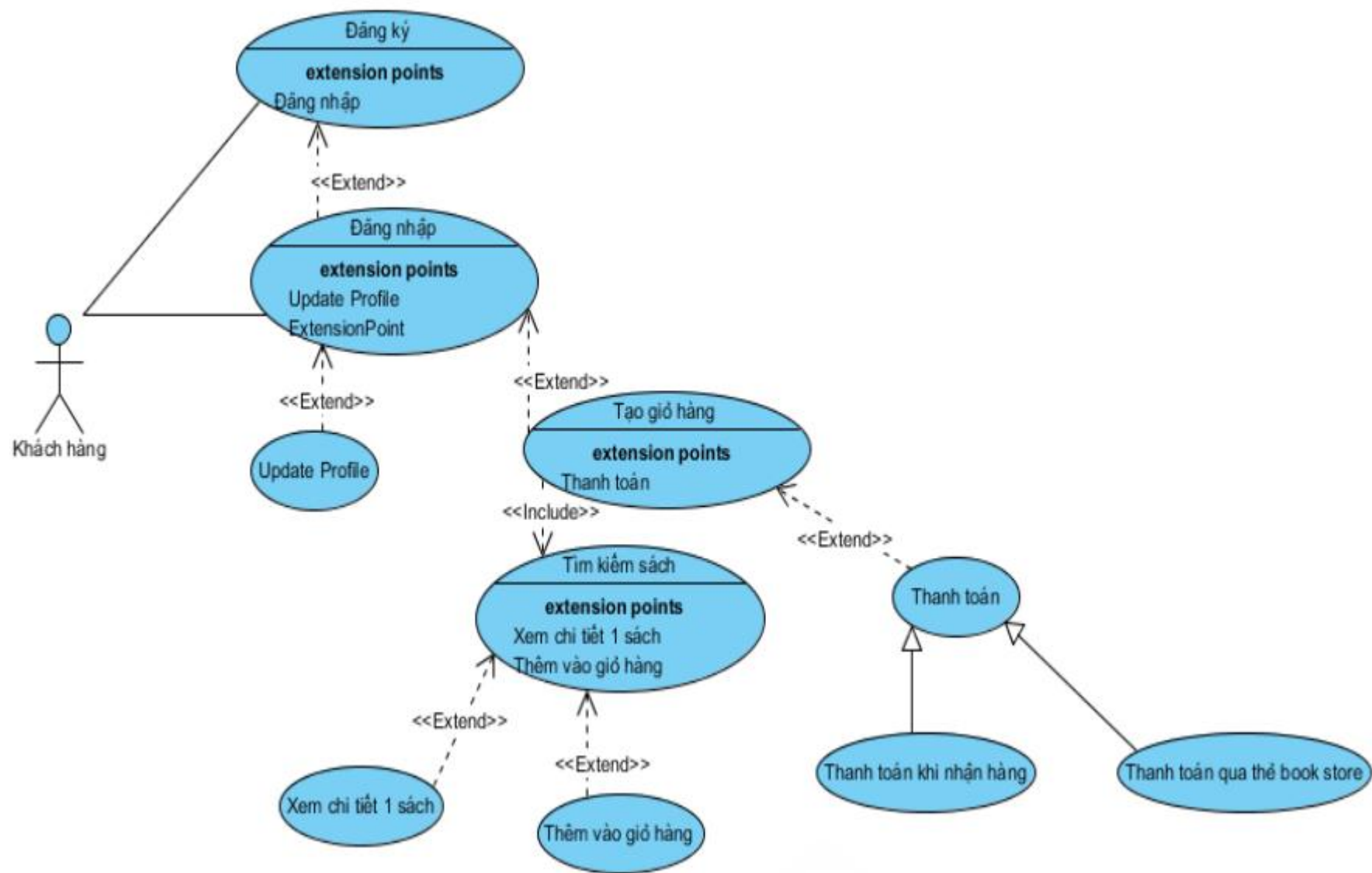
Ký hiệu của Use case
(Ca sử dụng)

I Mối quan hệ giữa các Use case

- **Association**: thường được dùng để mô tả mối quan hệ giữa Actor và Use case và giữa các Use case với nhau.
- **Include**: là quan hệ giữa các Use case với nhau, nó mô tả việc một Use case lớn được chia ra thành các Use case nhỏ để dễ cài đặt (module hóa) hoặc thể hiện sự dùng lại.
- **Extend**: Extend dùng để mô tả quan hệ giữa 2 Use case. Quan hệ Extend được sử dụng khi có một Use Case được tạo ra để bổ sung chức năng cho một Use case có sẵn và được sử dụng trong một điều kiện nhất định nào đó.
- **Generalization**: được sử dụng để thể hiện quan hệ thừa kế giữa các Actor hoặc giữa các Use Case với nhau.



| Ví dụ Biểu đồ ca sử dụng



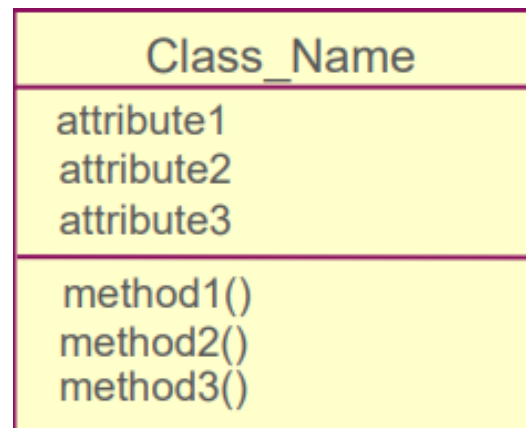
| 2.3.2 Biểu đồ lớp

❖ Ý nghĩa:

- Trong phương pháp hướng đối tượng, các đối tượng có chung một số thuộc tính và phương thức được nhóm thành một lớp. Tương tác giữa các đối tượng trong hệ thống sẽ được biểu diễn thông qua mối quan hệ giữa các lớp.
- **Biểu đồ lớp** là thể hiện mô hình tĩnh của hệ thống nhằm mô tả khung nhìn về một hệ thống bằng các khái niệm **lớp**, các **thuộc tính**, **phương thức** của lớp và mối quan hệ giữa chúng với nhau.

❖ Các thành phần của lớp:

- Tên lớp
- Các thuộc tính
- Các phương thức.



| Biểu diễn thuộc tính

phạmvi tên: kiểu sốĐỐItượng = mặcđịnh (Giá trịGiớihạn)

- *phạmvi* cho biết phạm vi truy nhập của thuộc tính. Có bốn là:
 - + : thuộc tính kiểu public
 - # : thuộc tính kiểu protected
 - : thuộc tính kiểu private.
 - ~ : thuộc tính được phép truy nhập tới từ các lớp trong cùng package.
- *tên*: là xâu ký tự biểu diễn tên thuộc tính.
- *kiểu*: là kiểu dữ liệu của thuộc tính.
- *sốĐỐItượng*: chỉ ra số đối tượng khai báo cho thuộc tính ứng với một *mặcđịnh*: là giá trị khởi đầu mặc định (nếu có) của thuộc tính.
- *GiátrịGiớihạn*: là giới hạn các giá trị cho thuộc tính (không bắt buộc)

I Biểu diễn phương thức

phạmvi tên(danhSáchThamsố): kiểuTralại {kiểuPhươngthức}

- *phạmvi* cho biết phạm vi truy nhập của phương thức. Có bốn là:
 - + : thuộc tính kiểu public
 - # : thuộc tính kiểu protected
 - : thuộc tính kiểu private.
 - ~ : thuộc tính được phép truy nhập tới từ các lớp trong cùng package.
- *tên*: là xâu ký tự biểu diễn tên phương thức.
- *kiểuTralại*: chỉ ra kiểu giá trị trả về của phương thức.
- *danhSáchThamsố*: biểu diễn danh sách các tham số trong khai báo của phương thức.
- *kiểuPhươngthức*: không bắt buộc, cho biết kiểu phương thức.
 - abstract: phương thức kiểu trừu tượng.
 - query: phương thức kiểu truy vấn.

| Ví dụ Biểu diễn lớp

Person
-name : String -birthDate : Date
+getName() : String +setName(name) : void +isBirthday() : boolean

Book
-title : String -authors : String[]
+getTitle() : String +getAuthors() : String[] +addAuthor(name)

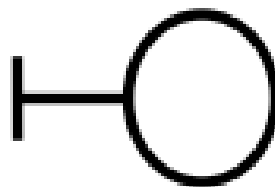
I Các kiểu lớp trong UML

UML định nghĩa ba kiểu lớp dựa trên vai trò của nó trong hệ thống bao gồm:

- **Lớp thực thể** (entity class): là lớp đại diện cho các thực thể thuộc miền quan tâm và chứa thông tin về các đối tượng xác định nào đó.
- **Lớp biên** (boundary class): là lớp nằm ở ranh giới giữa hệ thống với môi trường bên ngoài nhằm thực hiện vai trò nhận yêu cầu trực tiếp từ các tác nhân và chuyển các yêu cầu đó cho các lớp bên trong hệ thống.
- **Lớp điều khiển** (control class): thực hiện các chức năng điều khiển hoạt động của hệ thống tương ứng với các chức năng cụ thể nào đó của một nhóm các lớp biên hoặc nhóm các lớp thực thể.



Lớp thực thể



Lớp biên

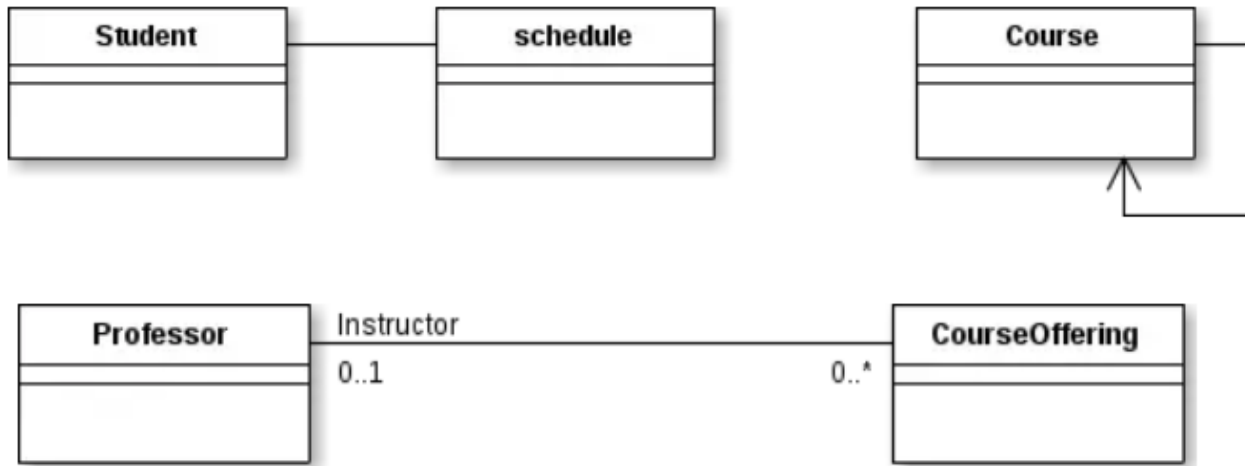


Lớp điều khiển

| Quan hệ giữa các lớp

- Liên kết (**Association**):

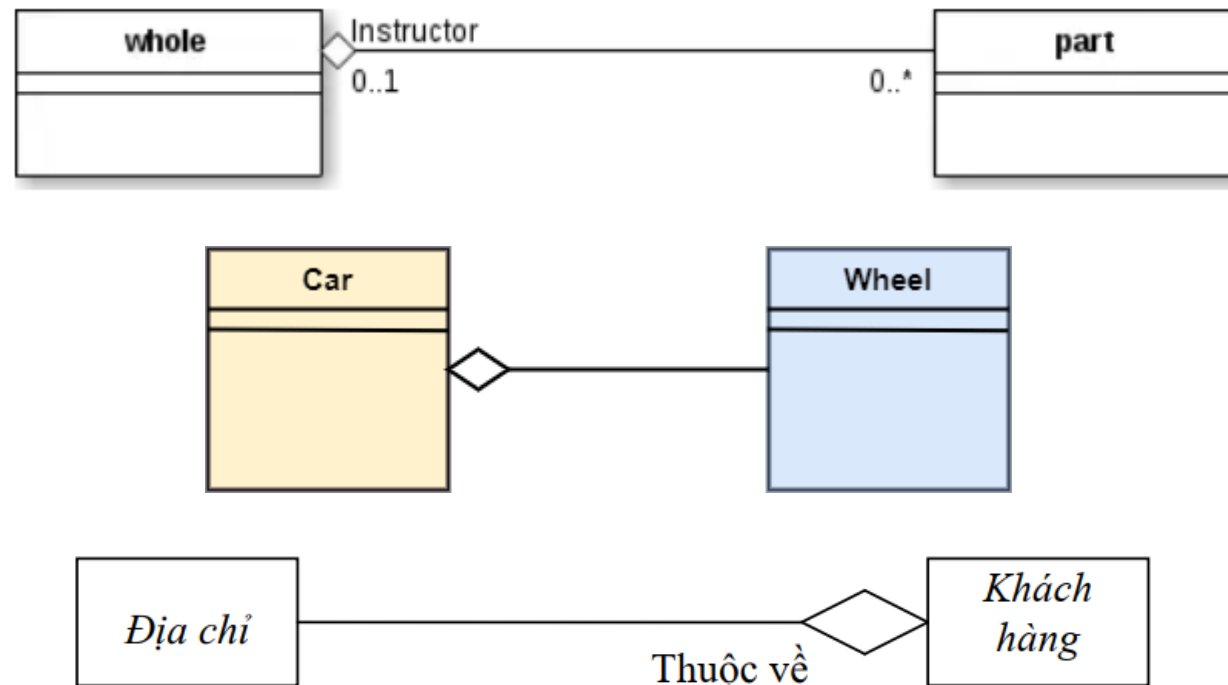
- Mỗi liên hệ ngữ nghĩa giữa hai hay nhiều lớp chỉ ra sự liên kết giữa các thể hiện của chúng.
- Bộ số quan hệ: là số lượng thể hiện của một lớp liên quan tới một thể hiện của lớp khác. Với mỗi liên kết, có hai bộ số quan hệ cho hai đầu của liên kết.



Biểu diễn	Ý nghĩa
	Unspecified
1	chính xác 1
0..*	0 hoặc nhiều
*	0 hoặc nhiều
1..*	1 hoặc nhiều
0..1	0 hoặc 1
2..4	Specified Range
2, 4..6	Multiple, Disjoint Ranges

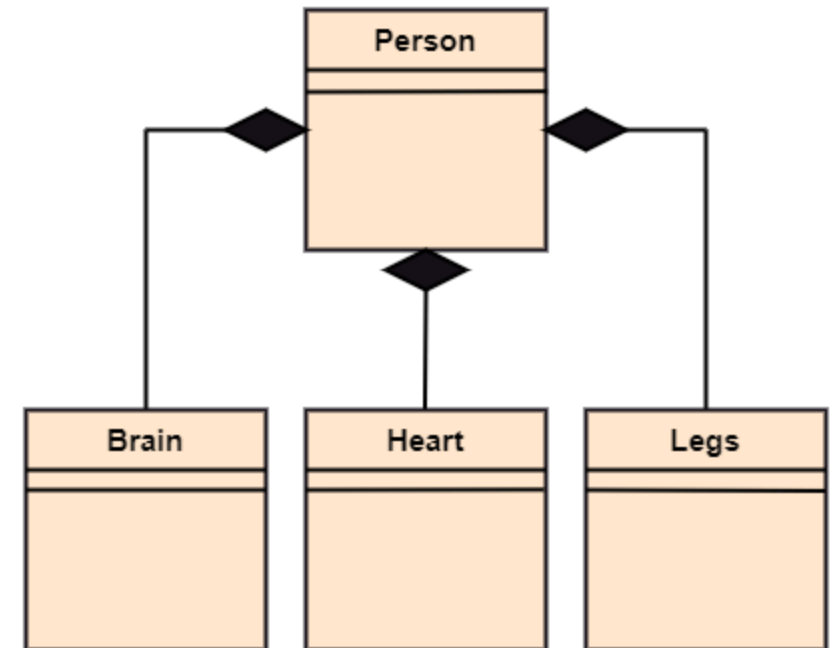
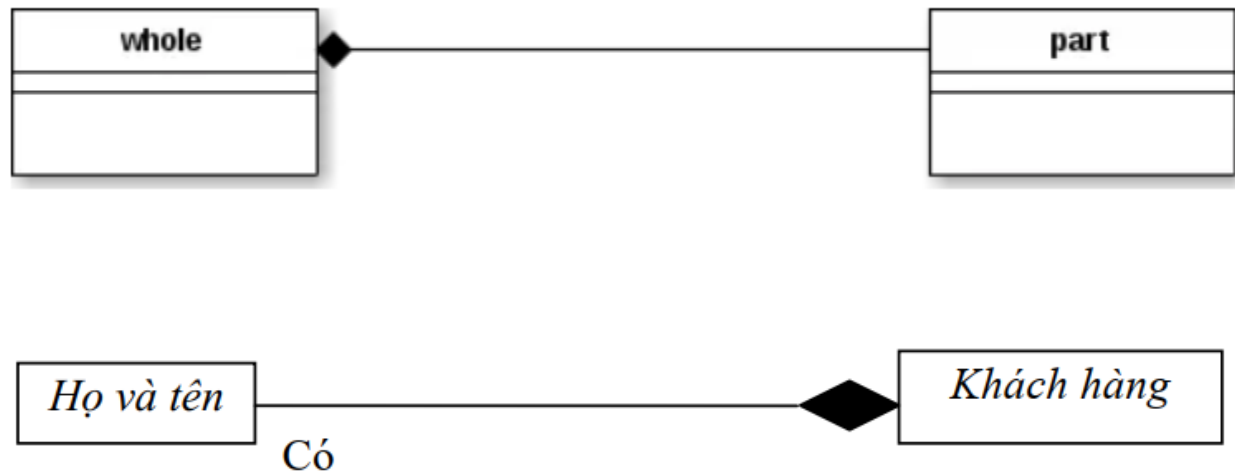
I Quan hệ giữa các lớp

- Kết tập (**Aggregation**): Là một dạng đặc biệt của liên kết mô hình hóa mối quan hệ toàn thể-bộ phận (whole-part) giữa đối tượng toàn thể và các bộ phận của nó.
- Kết tập là mối quan hệ “là một phần” (“is a part-of”).
- Bộ số quan hệ được biểu diễn giống như các liên kết khác.



I Quan hệ giữa các lớp

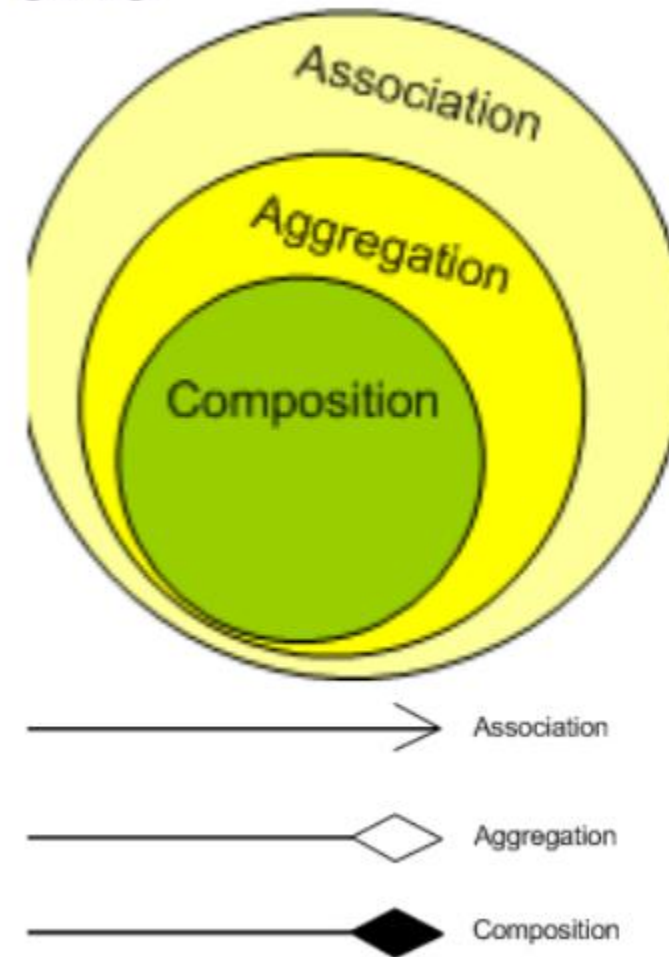
- Hợp thành (**Composition**) là: Một dạng của kết tập với quyền sở hữu mạnh và các vòng đời trùng khớp giữa hai lớp.
- Whole sở hữu Part, tạo và hủy Part.
- Part bị bỏ đi khi Whole bị bỏ, Part không thể tồn tại nếu Whole không tồn tại.



| Quan hệ giữa các lớp

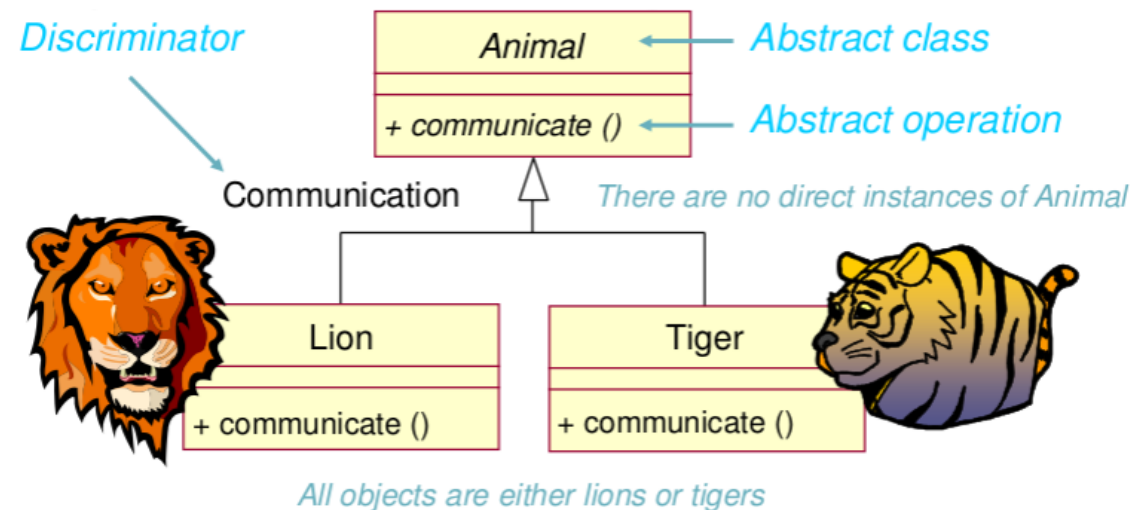
Association, Aggregation and Composition

- Mỗi quan hệ giữa các lớp (relationship)
 - Liên kết (Association)
 - Sử dụng (use-a)
 - Kết tập (Aggregation)
 - Strong association
 - has-a/is-a-part
 - Hợp thành (Composition)
 - Strong aggregation
 - Share life-time

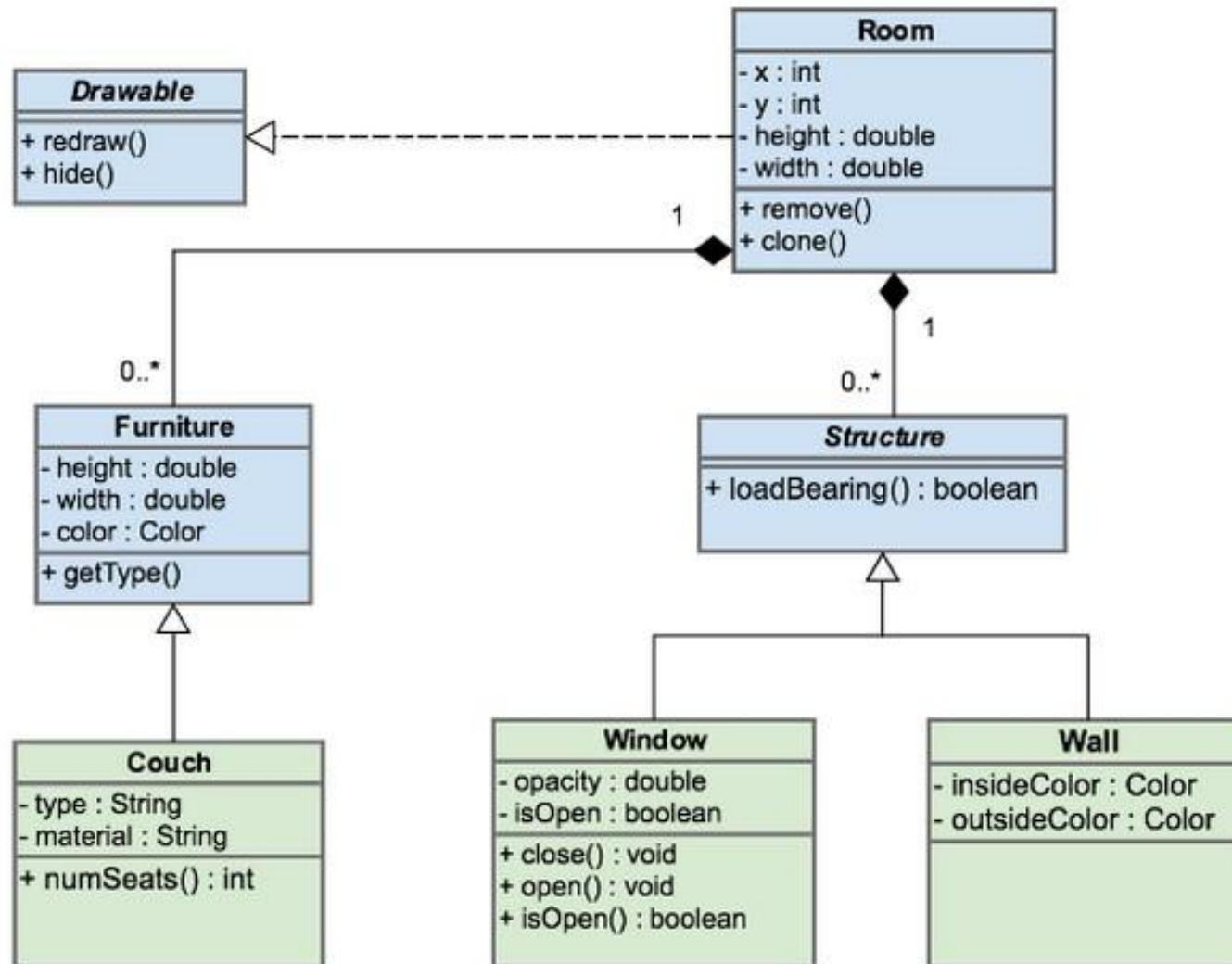


I Quan hệ giữa các lớp

- Tổng quát hóa (**Generalization**): Mỗi quan hệ giữa các lớp trong đó một lớp chia sẻ cấu trúc và/hoặc hành vi với một hoặc nhiều lớp khác.
- Xác định sự phân cấp về mức độ trừu tượng hóa trong đó lớp con kế thừa từ một hoặc nhiều lớp cha.
 - Đơn kế thừa (Single inheritance)
 - Đa kế thừa (Multiple inheritance)
- Là mối liên hệ “là một loại” (“is a kind of”).
- Lớp trừu tượng và lớp cụ thể (Abstract and Concrete Class).



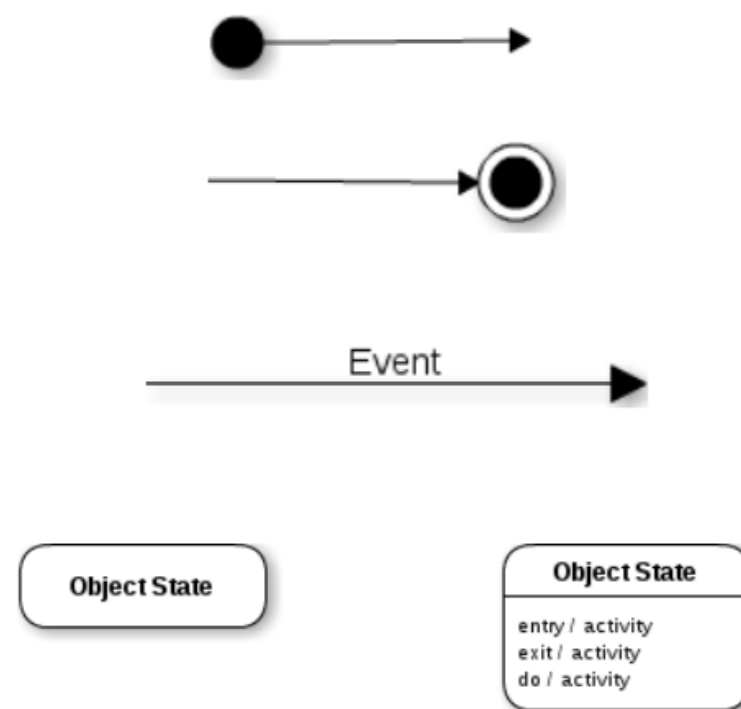
| Biểu đồ lớp



| 2.3.3 Biểu đồ trạng thái

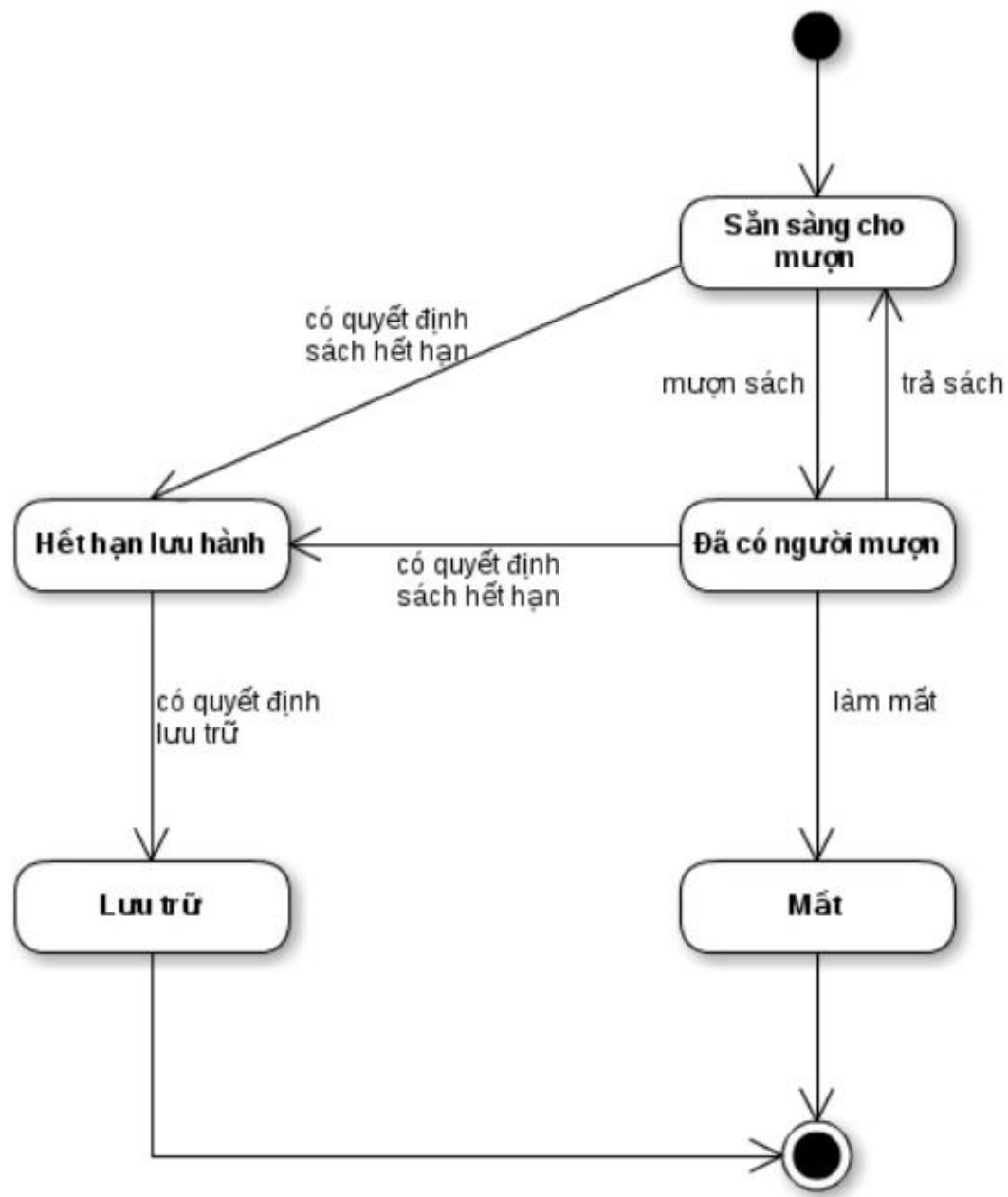
- **Biểu đồ trạng thái** được sử dụng để biểu diễn các trạng thái và sự chuyển tiếp giữa các trạng thái của các đối tượng trong một lớp.
- Các thành phần của Sơ đồ trạng thái:
 - **Trạng thái bắt đầu**: (Initial State).
 - **Trạng thái kết thúc**: (Final State):

Trong biểu đồ, đường mũi tên chỉ ra sự biến đổi từ một trạng thái sang trạng thái khác.
 - **Sự kiện** (Event) hoặc **Chuyển đổi** (Transition).
 - **Trạng thái đối tượng** (State)



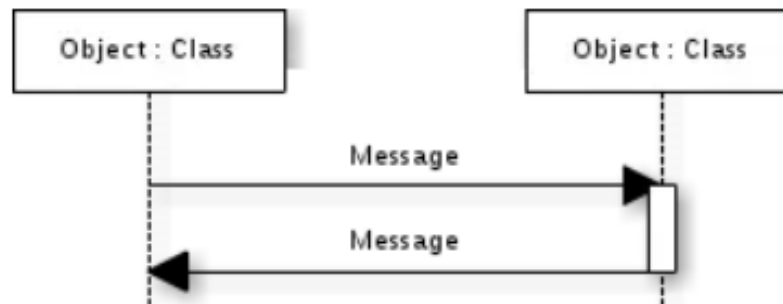
| Biểu đồ trạng thái – Ví dụ

- Biểu đồ trạng thái của Lớp Sách trong thư viện.



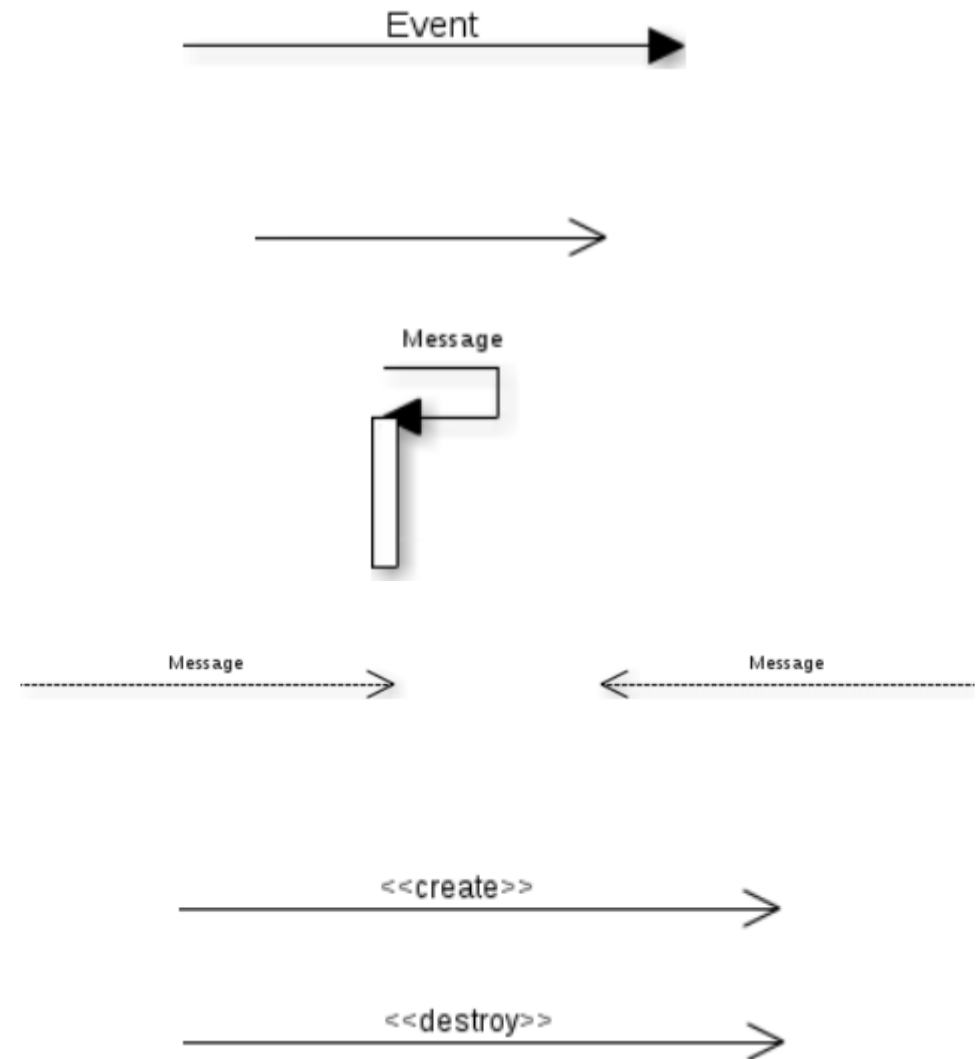
| 2.3.4 Biểu đồ tuần tự (Sequence Diagram)

- **Biểu đồ tuần tự** là biểu đồ dùng để xác định các trình tự diễn ra sự kiện của một nhóm đối tượng nào đó. Nó miêu tả chi tiết các thông điệp được gửi và nhận giữa các đối tượng đồng thời cũng chú trọng đến việc trình tự về mặt thời gian gửi và nhận các thông điệp đó.
- Các thành phần của sơ đồ tuần tự:
 - **Đối tượng** (object hoặc class): biểu diễn bằng các hình chữ nhật.
 - **Đường đời đối tượng** (Lifelines): biểu diễn bằng các đường gạch rời thẳng đứng bên dưới các đối tượng.
 - **Thông điệp** (Message): biểu diễn bằng các đường mũi tên.
 - **Xử lý bên trong đối tượng** (biểu diễn bằng các đoạn hình chữ nhật rỗng nối với các đường đời đối tượng).

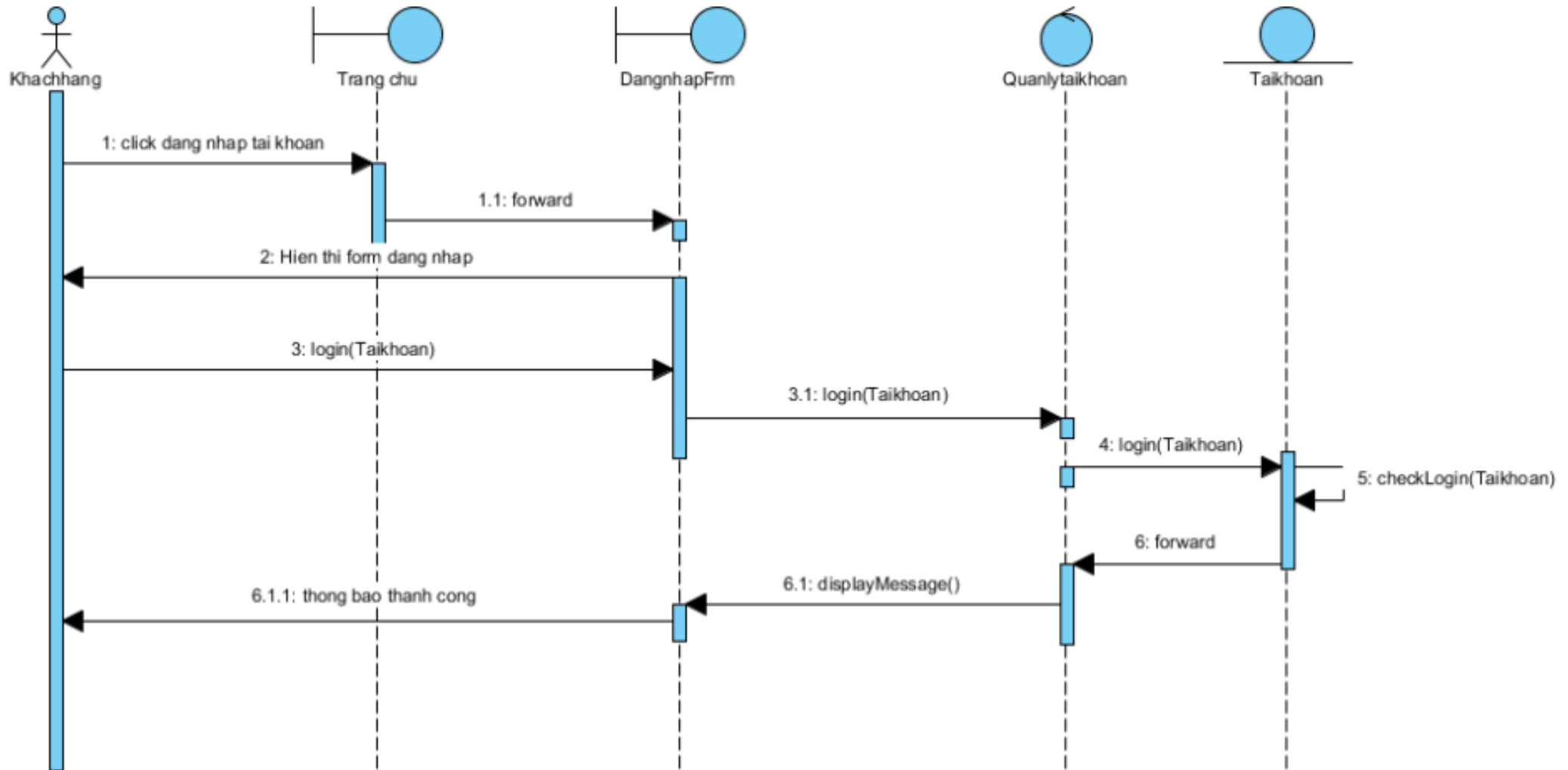


I Biểu đồ tuần tự – Các loại thông điệp

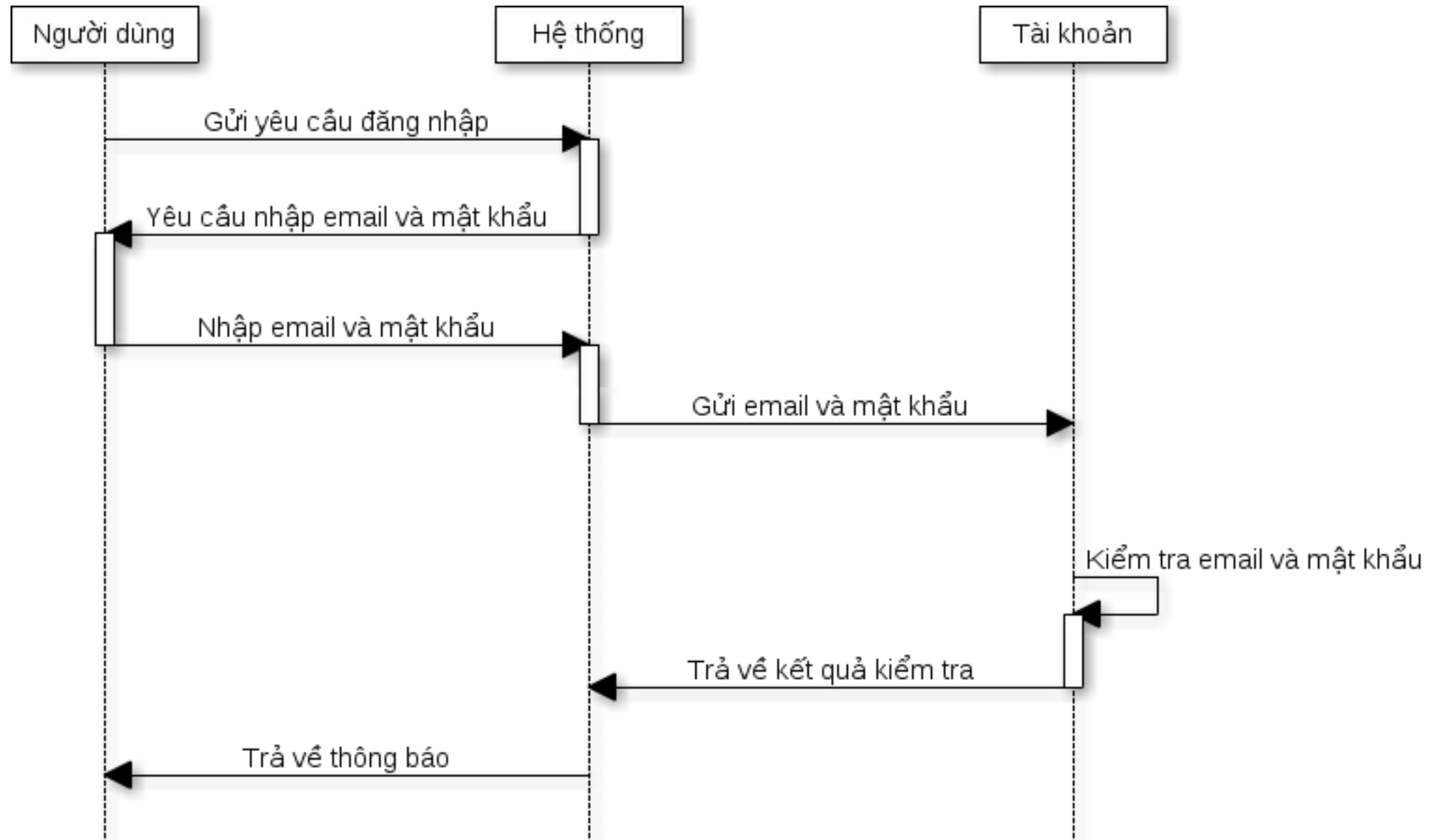
- **Thông điệp đồng bộ (Synchronous Message)** Thông điệp đồng bộ cần có một request trước hành động tiếp theo.
- **Thông điệp không đồng bộ (Asynchronous Message)** Thông điệp không đồng bộ không cần có một request trước hành động tiếp theo.
- **Thông điệp chính mình (Self Message)** Là thông điệp mà đối tượng gửi cho chính nó để thực hiện các hàm nội tại.
- **Thông điệp trả lời hoặc trả về (Reply or Return Message)**: Là thông điệp trả lời lại khi có request hoặc sau khi kiểm tra tính đúng đắn của một điều kiện nào đó.
- **Thông điệp tạo mới (Create Message)**: Là thông điệp được trả về khi tạo mới một đối tượng.
- **Thông điệp xóa (Delete Message)**: Là thông điệp được trả về khi xóa một đối tượng



| Biểu đồ tuần tự – Ví dụ



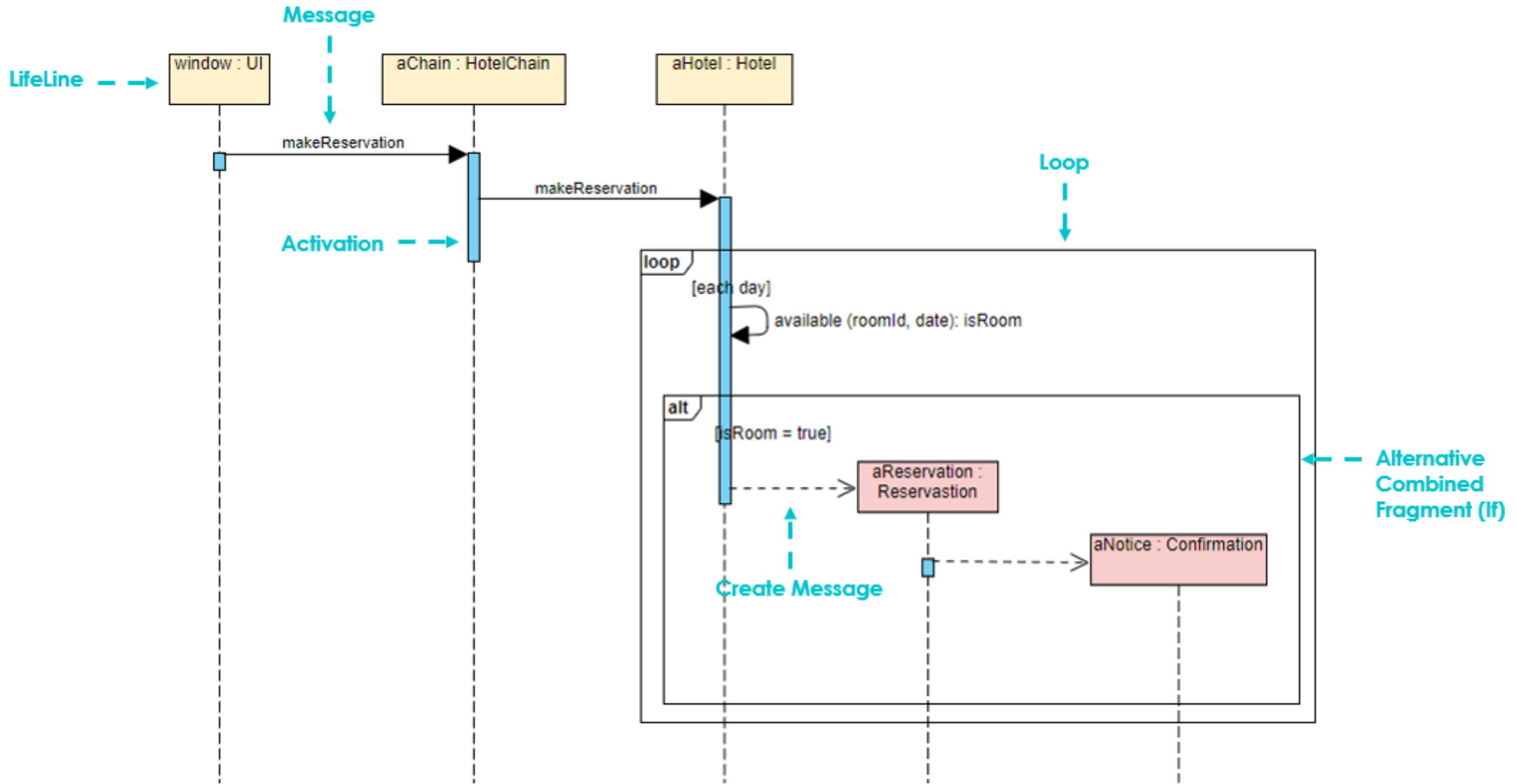
| Biểu đồ tuần tự – Ví dụ



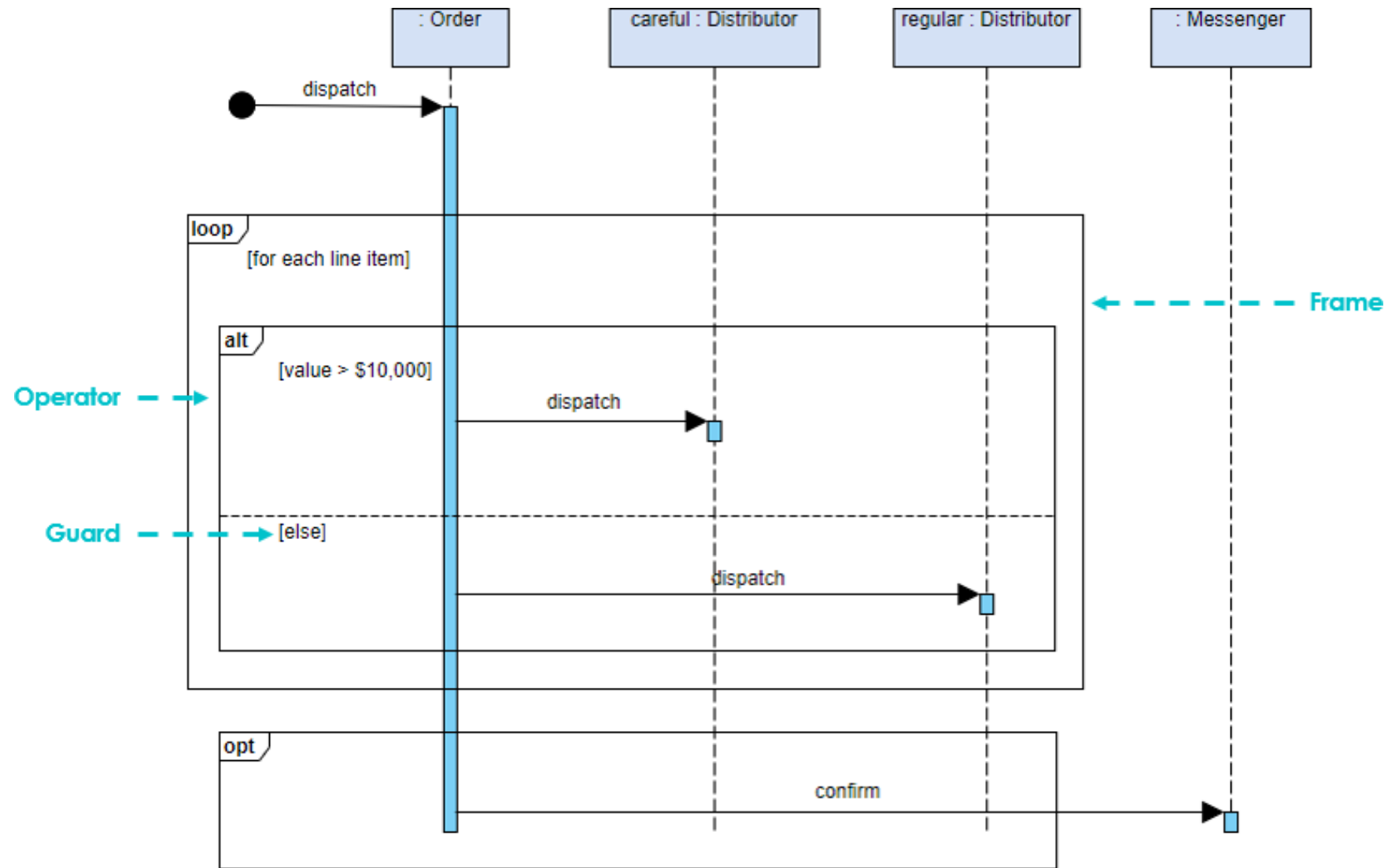
| Các khối kết hợp trong UML (Combined Fragments)

- **alt** (Alternative Fragment): Mô tả các kịch bản thay thế, giống switch-case trong lập trình. Trong một khối alt, chỉ một trong các nhánh sẽ được thực thi tùy theo điều kiện được đặt trong dấu [].
- **opt** (Optional Fragment): Tương tự if, chỉ có một nhánh. Nếu điều kiện trong dấu [] là đúng, khối lệnh trong opt sẽ được thực hiện, ngược lại thì không làm gì.
- **loop** (Loop Fragment): Diễn tả vòng lặp với số lần lặp tối thiểu/tối đa. Có thể chỉ định số lần lặp trong dấu (min..max), hoặc nếu không có giới hạn trên, dùng ký hiệu (*).
- **break** (Break Fragment): Tương tự break trong lập trình. Nếu điều kiện trong break đúng, các thao tác còn lại trong khối bao quanh sẽ bị hủy bỏ.
- **par** (Parallel Fragment): Các khối lệnh sẽ chạy song song.

| Biểu đồ tuần tự – Ví dụ

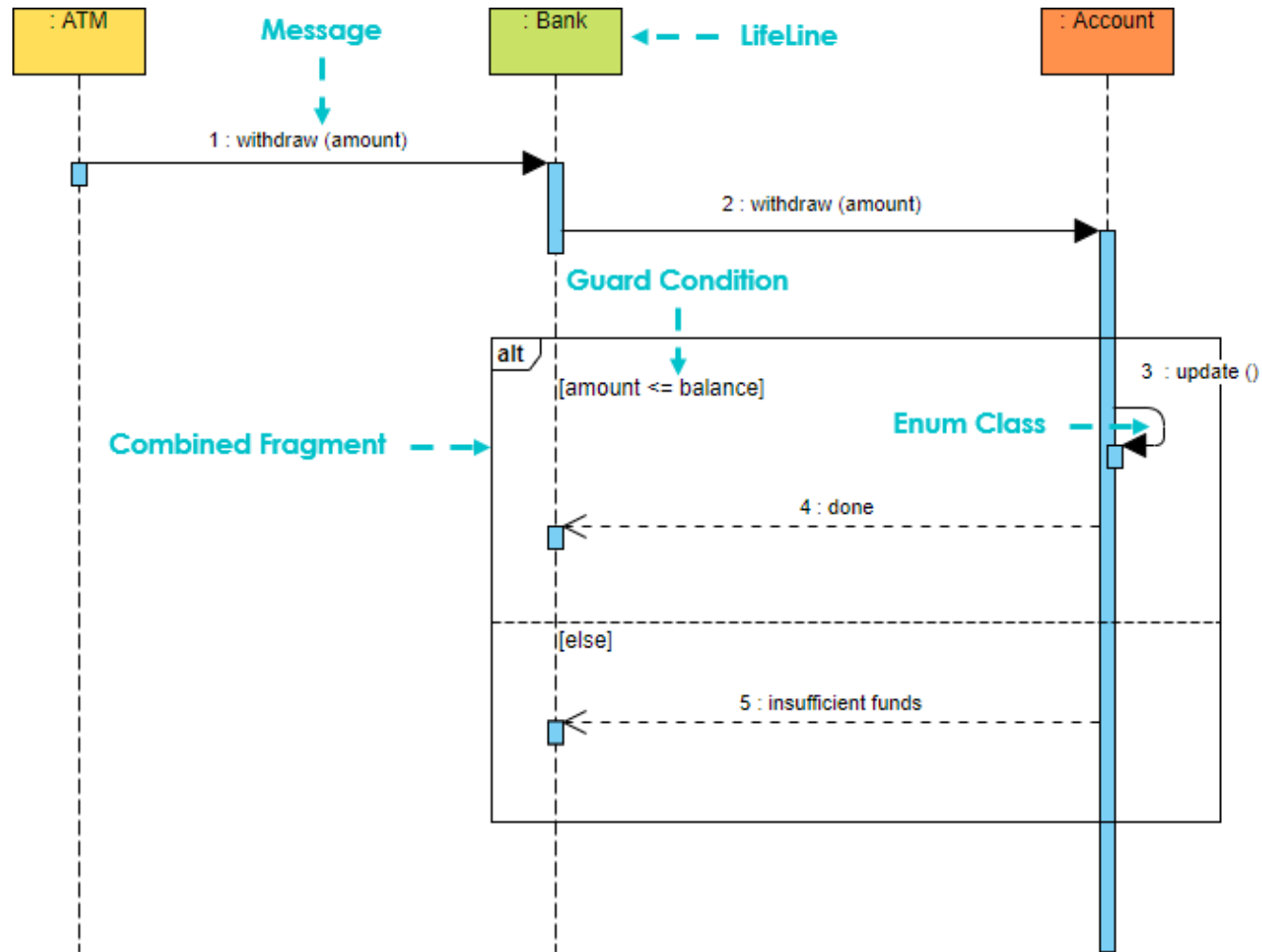


| Biểu đồ tuần tự – Ví dụ



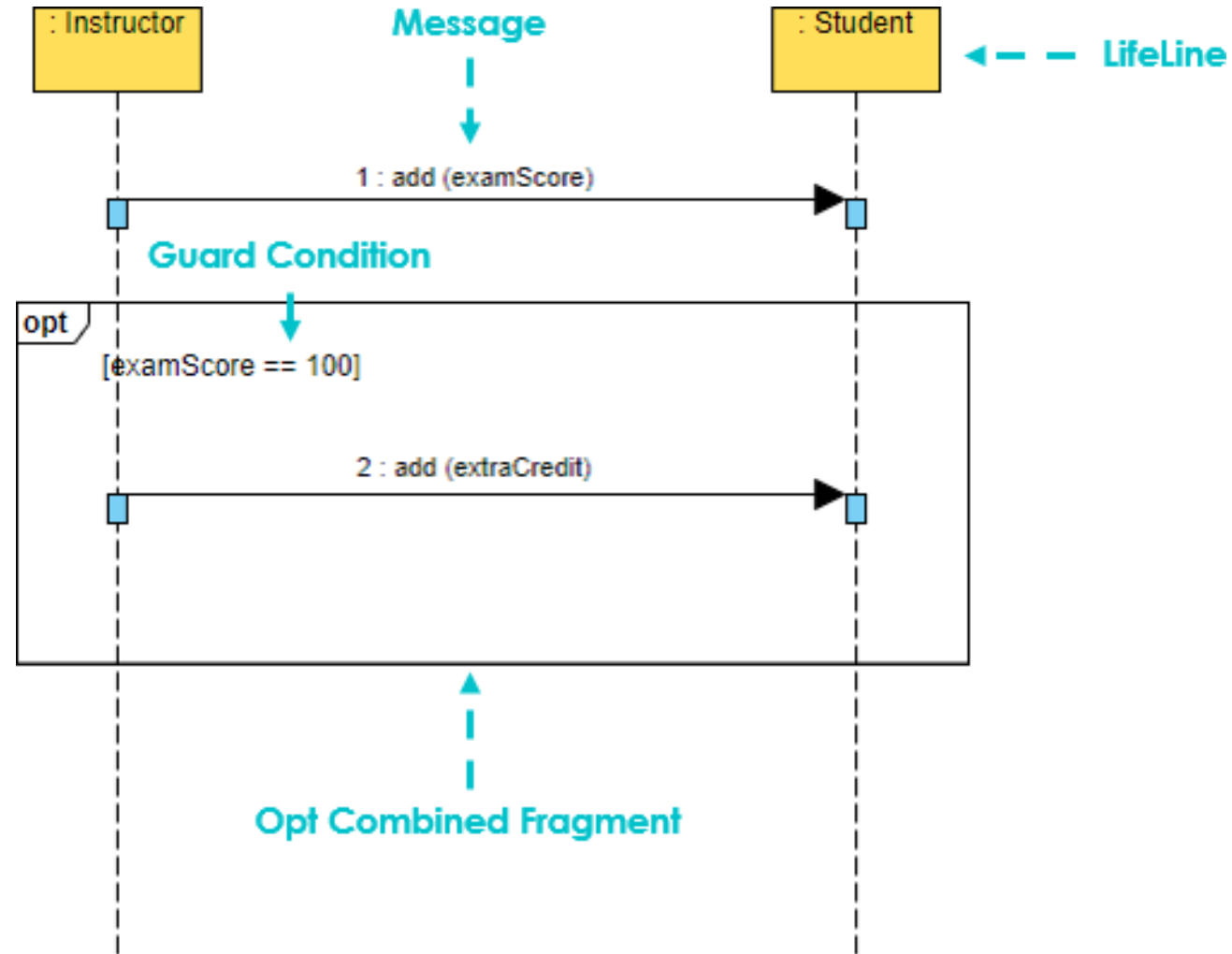
| Biểu đồ tuần tự

Ví dụ với alt



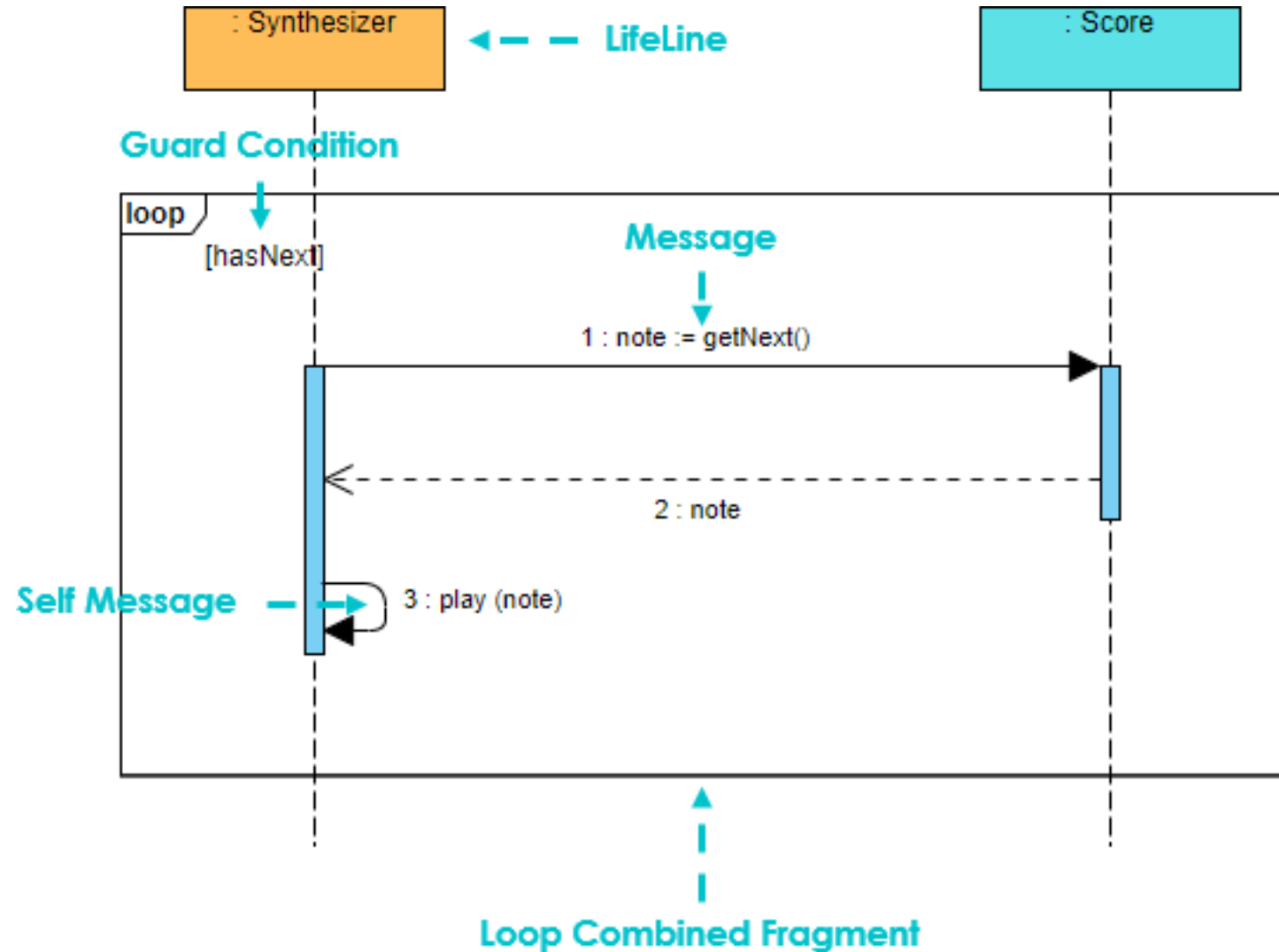
| Biểu đồ tuần tự

Ví dụ với **opt**



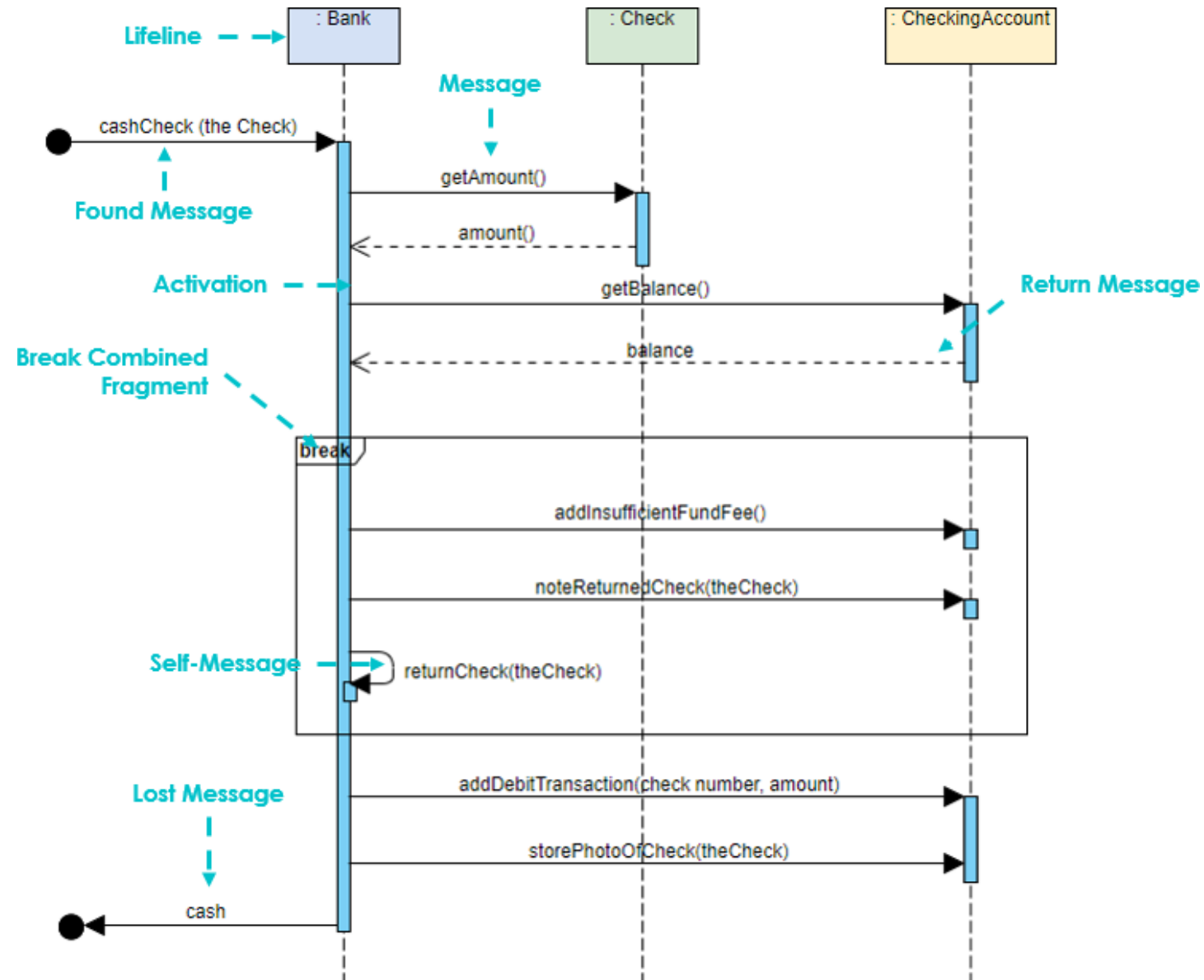
| Biểu đồ tuần tự

Ví dụ với loop



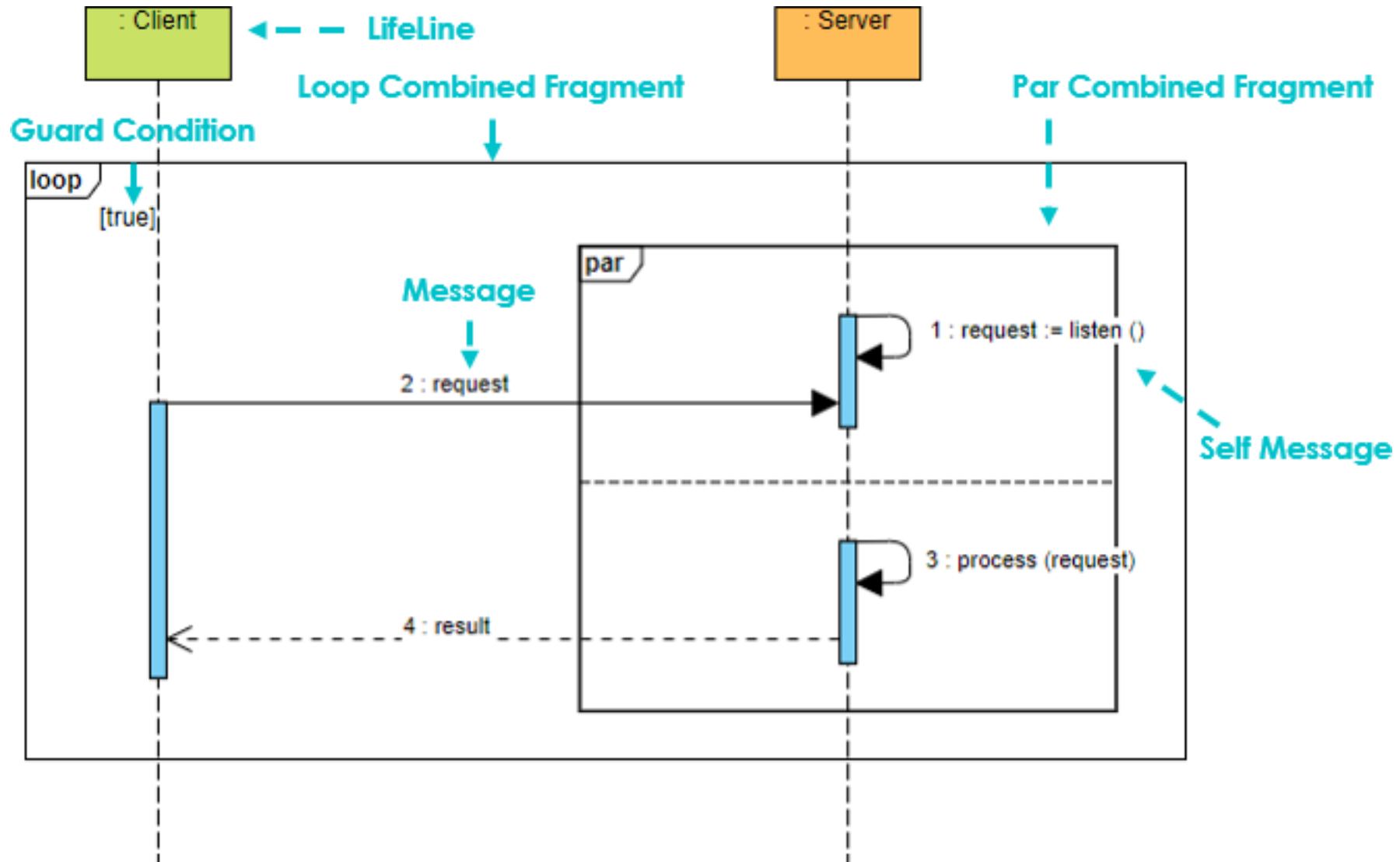
| Biểu đồ tuần tự

Ví dụ với break



| Biểu đồ tuần tự

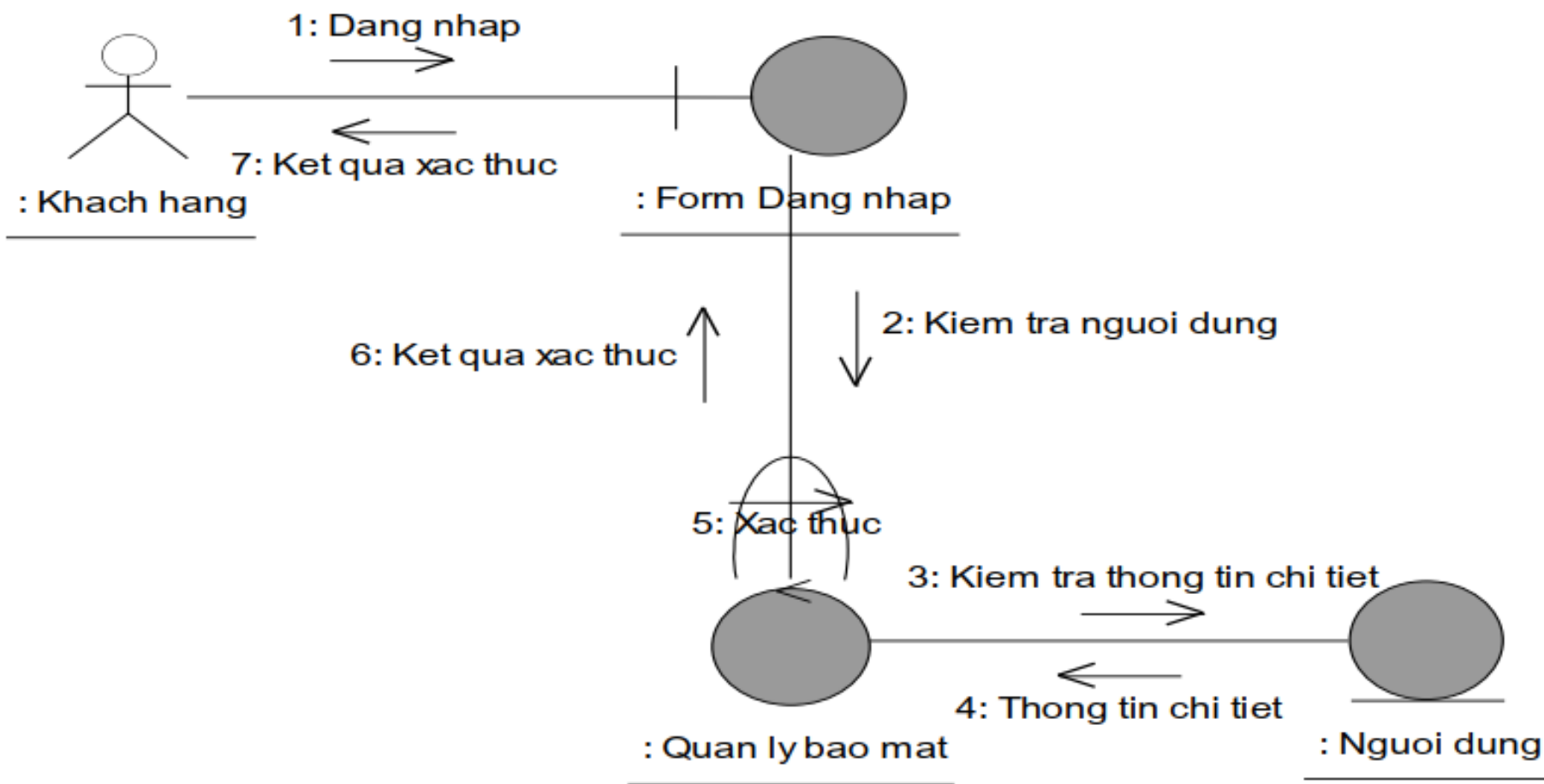
Ví dụ với **par**



| 2.3.5 Biểu đồ giao tiếp

- **Biểu đồ giao tiếp** là loại biểu đồ tương tác biểu diễn mối quan hệ giữa các đối tượng, giữa đối tượng và tác nhân. Biểu đồ giao tiếp cũng có các thông điệp với nội dung tương tự như trong biểu đồ tuần tự. Tuy nhiên, các đối tượng được đặt một cách tự do trong không gian của biểu đồ và không có đường vòng đời cho các đối tượng; các thông điệp được đánh số thể hiện thứ tự thời gian.
- Tập ký hiệu UML cho biểu đồ giao tiếp:
 - **Các đối tượng**: được biểu diễn bởi các hình chữ nhật, bên trong là tên của đối tượng. Cách viết chung của đối tượng là: **tên đối tượng: tên lớp**. Trong biểu đồ giao tiếp, các đối tượng tham gia tương tác luôn xuất hiện tại một vị trí xác định.
 - **Các liên kết**: hai đối tượng có tương tác sẽ có một liên kết nối hai đối tượng đó.
 - **Các thông điệp**: được biểu diễn bằng các mũi tên hướng từ đối tượng gửi sang đối tượng nhận bên cạnh liên kết giữa hai đối tượng đó. Trong biểu đồ giao tiếp, các thông điệp được đánh số theo thứ tự xuất hiện trong kịch bản mô tả ca sử dụng tương ứng.

| Biểu đồ giao tiếp – Ví dụ



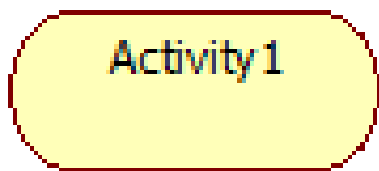
| 2.3.6 Biểu đồ hoạt động

- **Biểu đồ hoạt động** biểu diễn các hoạt động và sự đồng bộ, chuyển tiếp các hoạt động của hệ thống trong một lớp hoặc kết hợp giữa các lớp với nhau trong một chức năng cụ thể.
- Biểu đồ hoạt động có thể được sử dụng cho nhiều mục đích khác nhau, ví dụ như:
 - Để xác định các hành động phải thực hiện trong phạm vi một phương thức.
 - Để xác định công việc cụ thể bên trong của một đối tượng.
 - Để chỉ ra một nhóm hành động liên quan của các đối tượng được thực hiện như thế nào và chúng sẽ ảnh hưởng thế nào đến những đối tượng xung quanh.

| Biểu đồ hoạt động

- **Hoạt động** (Activity): là một quy trình được định nghĩa rõ ràng, có thể được thực hiện bởi một phương thức hoặc một nhóm đối tượng. Hoạt động được thể hiện bằng hình chữ nhật tròn cạnh.
- **Thanh đồng bộ hóa** (Synchronisation bar): cho phép ta mở ra hoặc là đóng lại các nhánh chạy song song trong tiến trình.
- **Điều kiện** (Guard Condition): các biểu thức logic có giá trị hoặc đúng hoặc sai. Điều kiện được thể hiện trong ngoặc vuông, ví dụ: [Customer existing].
- **Các luồng** (swimlane): Mỗi biểu đồ hoạt động có thể biểu diễn sự phối hợp hoạt động trong nhiều lớp khác nhau. Khi đó mỗi lớp được phân tách bởi một luồng (swimlane) riêng biệt và các luồng này được biểu diễn đơn giản là các ô khác nhau trong biểu đồ.

| Biểu đồ hoạt động



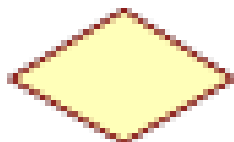
Hoạt động



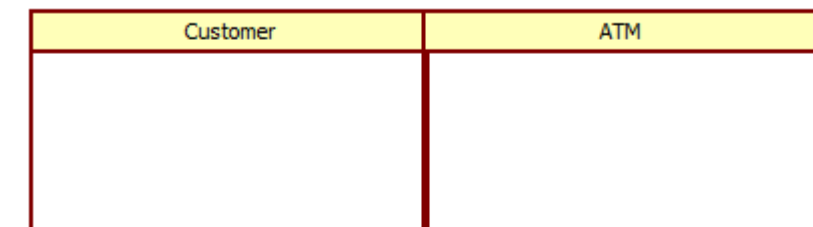
Trạng thái khởi đầu



Trạng thái kết thúc



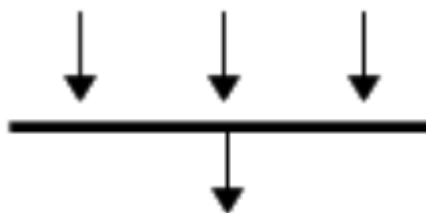
Rẽ nhánh



Swimlane



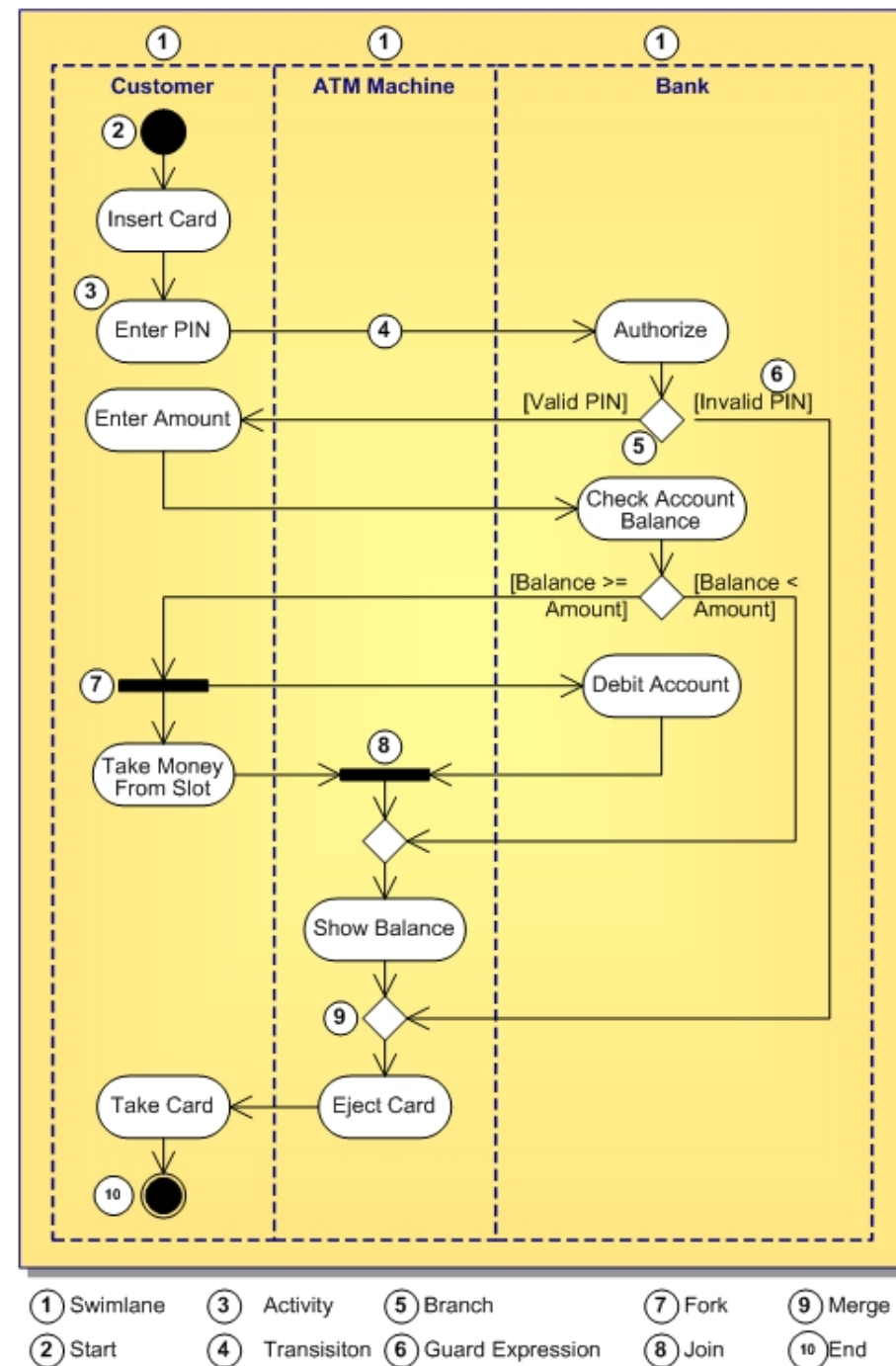
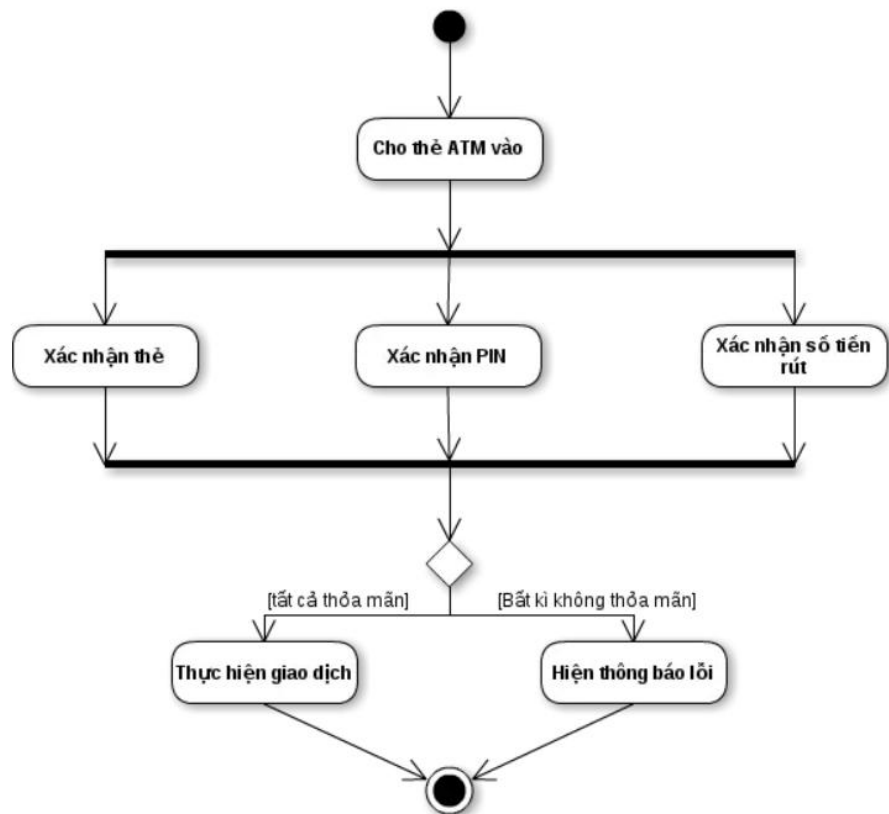
Sự chuyển tiếp



Thanh đồng bộ

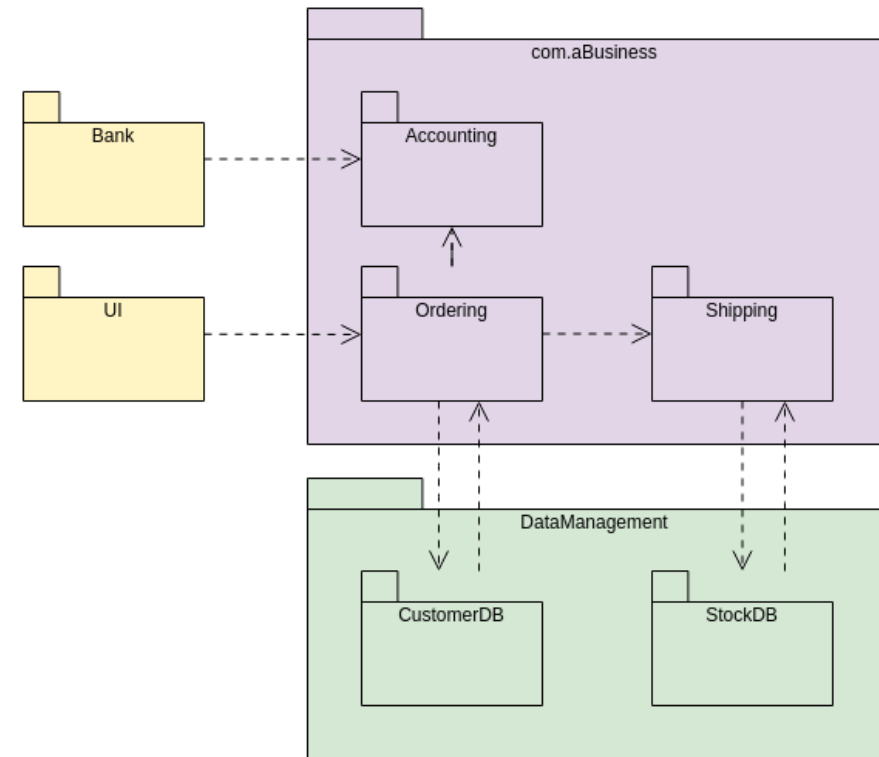
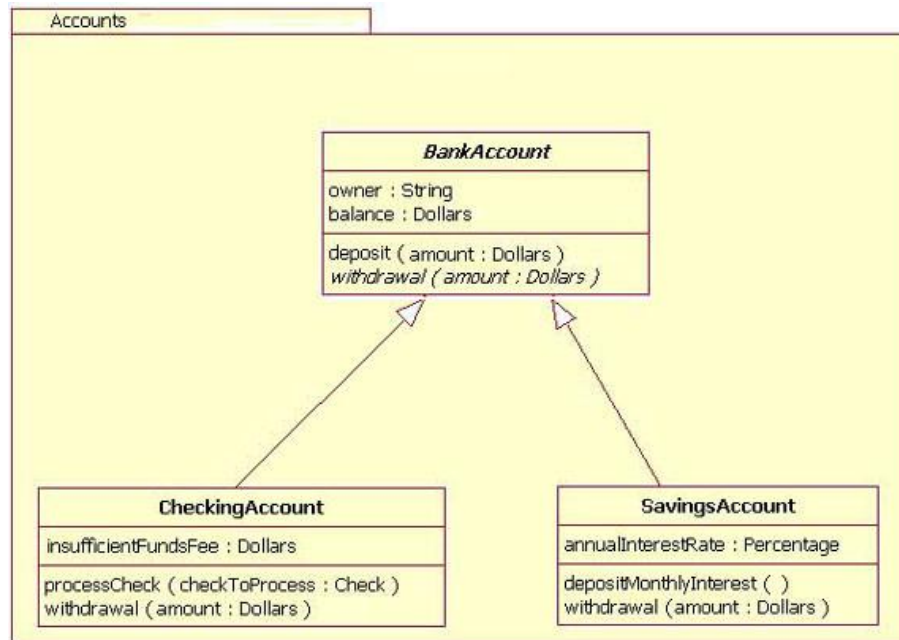
| Biểu đồ hoạt động – Ví dụ

- Biểu đồ hoạt động của hoạt động rút tiền từ ATM



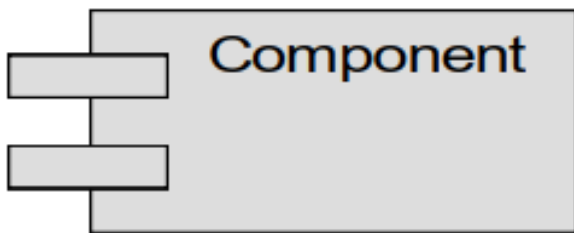
| 2.3.7 Biểu đồ gói (Package Diagram)

- Các **gói** trong **UML** được sử dụng để nhóm các phần tử và cung cấp không gian tên cho các phần tử được nhóm. Một gói có thể chứa các gói khác, do đó cung cấp một tổ chức phân cấp các gói.



| 2.3.8 Biểu đồ thành phần

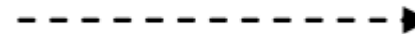
- **Biểu đồ thành phần** được sử dụng để biểu diễn các thành phần phần mềm cấu thành nên hệ thống.
- Mỗi thành phần có thể coi như một phần mềm cung cấp một số chức năng dưới dạng một hộp đen để xây dựng phần mềm lớn hơn, ví dụ như JavaBean hay các thư viện DLL...



Thành phần

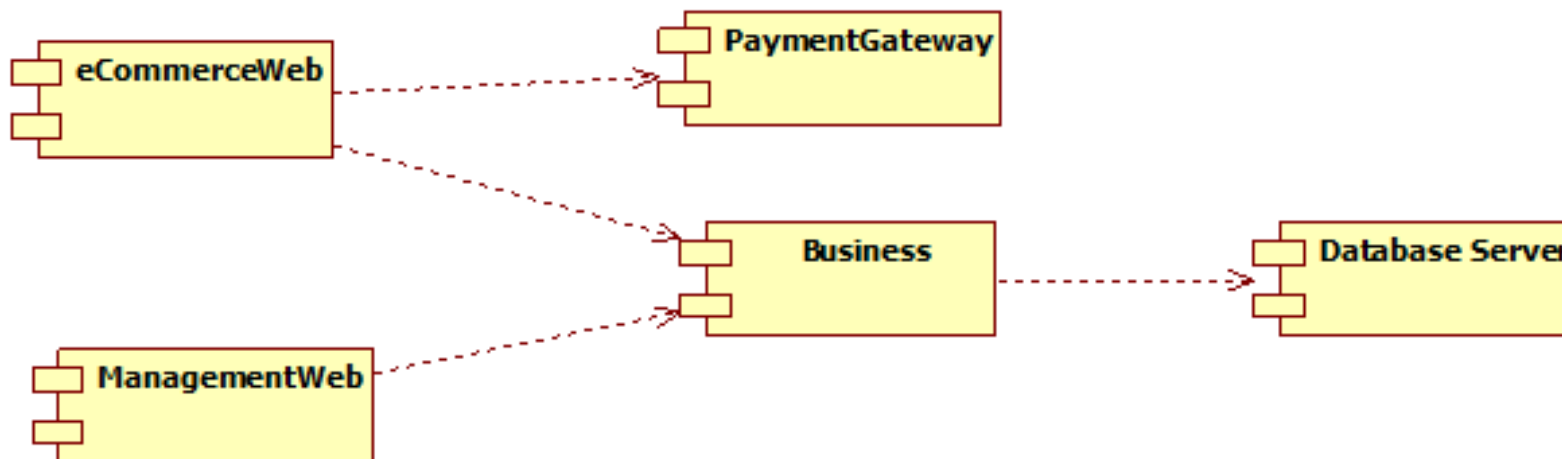


Giao tiếp với thành phần



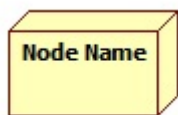
Quan hệ phụ thuộc
giữa các thành phần

| Biểu đồ thành phần – Ví dụ



| 2.3.9 Biểu đồ triển khai

- **Biểu đồ triển khai** là một loại biểu đồ thể hiện kiến trúc thực thi của hệ thống, bao gồm các nút như môi trường thực thi phần cứng hoặc phần mềm và phần mềm trung gian kết nối chúng.
- Biểu đồ triển khai thường được sử dụng để hình dung phần cứng và phần mềm vật lý của hệ thống.
- **Node**: Node là một thành phần vật lý, nó có thể là thiết bị phần cứng hoặc một môi trường nào đó mà các thành phần phần mềm được thực hiện.
- **Relationship**: Sử dụng quan hệ **Association** và **Dependence** để thể hiện mối quan hệ giữa các node với nhau.



Node

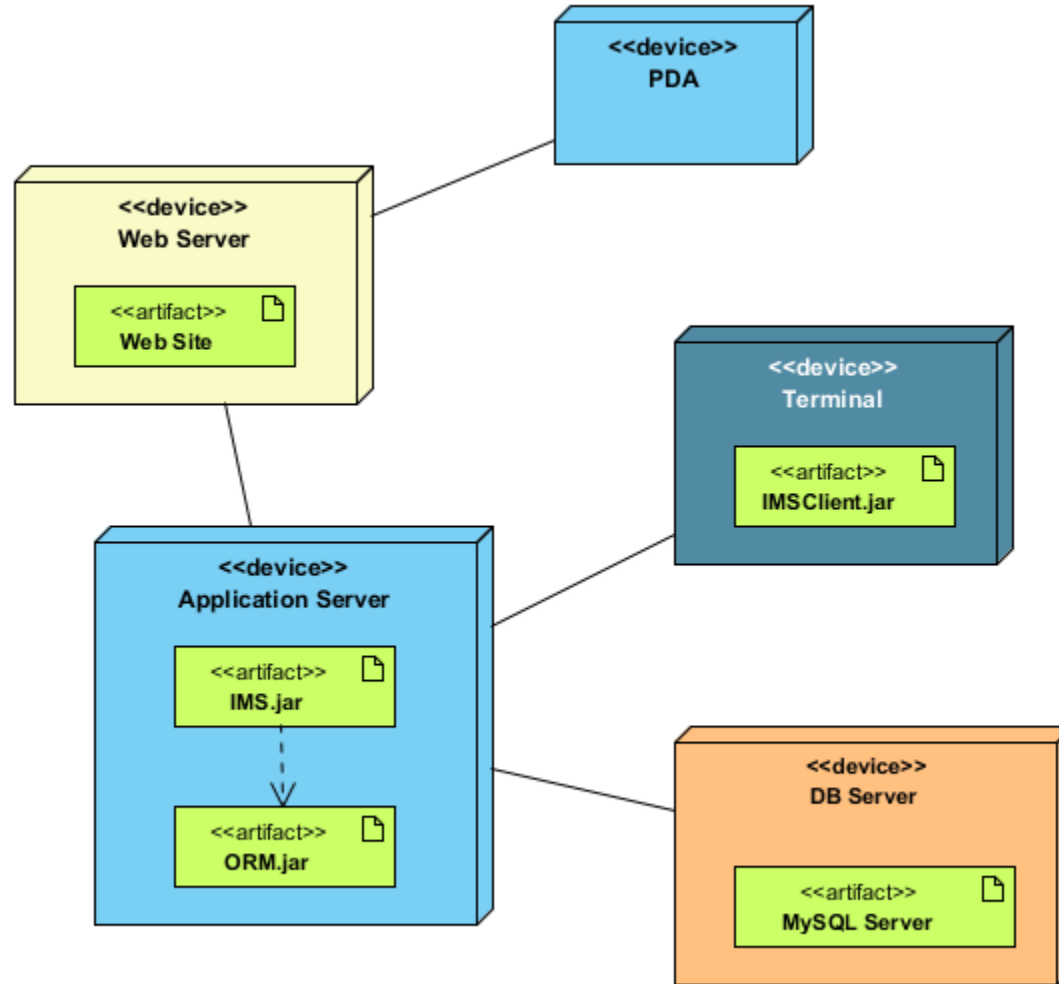


Ký hiệu về Association



Ký hiệu về Dependence

| Biểu đồ triển khai – Ví dụ



| BÀI TẬP

1. Trong hệ quản lý Bán hàng Sách online, trước hết khách hàng phải làm thủ tục đăng ký thành viên và khi đó hệ thống sẽ cung cấp một account để đăng nhập. Sau khi đăng nhập khách hàng có thể tìm kiếm, tạo giỏ hàng, đăng ký mua hàng, thanh toán... Hãy xác định các ca sử dụng và vẽ biểu đồ quan hệ giữa các ca sử dụng này.
2. Xây dựng biểu đồ tuần tự và biểu đồ giao tiếp cho đăng nhập và tạo giỏ hàng được trình bày trong câu 1.
3. Xây dựng biểu đồ hoạt động ca sử dụng đăng ký mua hàng được trình bày trong câu 1.
4. Hãy xác định các lớp, mối quan hệ giữa các lớp và xây dựng biểu đồ lớp cho nội dung trong câu 1.

Lưu ý:

- Đây là bài tập cá nhân không phải bài tập nhóm.
- Có thể chọn chủ đề khác như: Quản lý thư viện, Quản lý tour du lịch, Quản lý tín chỉ...

